



VITE & GOURMAND

Documentation technique

Projet ECF

Titre professionnel

Développeur Web et Web Mobile

Yoann LAMOUILLE

2026

Table des matières

<u>1</u>	<u>Présentation de l'architecture technique.....</u>	<u>3</u>
1.1	Architecture générale.....	3
1.2	Organisation MVC.....	3
1.3	Technologies utilisées.....	5
<u>2</u>	<u>Conception et gestion des données.....</u>	<u>6</u>
2.1	Modèle conceptuel de données.....	6
2.2	Base de données relationnelle MySQL.....	8
2.3	Accès aux données avec PDO.....	9
2.4	Base de données NoSQL MongoDB.....	10
2.5	Initialisation et jeu de données.....	13
<u>3</u>	<u>Architecture et fonctionnement de l'application.....</u>	<u>14</u>
3.1	Architecture MVC.....	14
3.2	Utilisation des entités.....	15
3.3	Routage et contrôleurs.....	17
3.4	Authentification et gestion des rôles.....	19
3.5	Gestion des menus.....	20
3.6	Gestion des commandes.....	23
3.7	Suivi des commandes et gestion des statuts.....	25
3.8	Gestion des e-mails transactionnels.....	27
3.9	Gestion des avis clients.....	29
3.10	Gestion des utilisateurs et des employés.....	31
<u>4</u>	<u>Sécurité de l'application.....</u>	<u>32</u>
4.1	Sécurisation de l'authentification et des mots de passe.....	33
4.2	Gestion sécurisée des sessions et des autorisations.....	34
4.3	Sécurisation des données saisies et des requêtes SQL.....	37
4.4	Protection des données sensibles et configuration de l'environnement.....	38
4.5	Sécurisation des fichiers téléversés.....	40
4.6	Réinitialisation sécurisée du mot de passe.....	42
<u>5</u>	<u>Tests et validation de l'application.....</u>	<u>43</u>
5.1	Tests unitaires avec PHPUnit.....	43
5.2	Exécution et résultats des tests.....	44
5.3	Tests fonctionnels manuels.....	45
<u>6</u>	<u>Déploiement et mise en production.....</u>	<u>47</u>
6.1	Environnement local avec WampServer.....	47
6.2	Conteneurisation avec Docker.....	48
6.3	Déploiement sur Heroku.....	50
6.4	Configuration de l'environnement de production.....	52
<u>7</u>	<u>Conclusion.....</u>	<u>53</u>

1 Présentation de l'architecture technique

L'application Vite & Gourmand a été développée sous la forme d'une application web dynamique organisée selon une architecture MVC (Modèle-Vue-Contrôleur). Cette organisation permet de séparer la gestion des données, le traitement des actions de l'utilisateur et l'affichage des différentes pages.

Le projet repose principalement sur PHP pour le développement côté serveur, MySQL pour le stockage des données relationnelles et MongoDB pour certaines données non relationnelles de l'application. L'interface utilisateur utilise HTML, CSS, JavaScript ainsi que Bootstrap.

L'environnement de développement a également été conteneurisé avec Docker afin de faciliter l'installation et l'exécution des différents services nécessaires à l'application.

Cette documentation présente l'organisation technique du projet, la conception et la gestion des données, le fonctionnement des principaux composants de l'application ainsi que la configuration de l'environnement et du déploiement.

1.1 Architecture générale

L'application est organisée en plusieurs composants ayant chacun une responsabilité distincte. Les requêtes de l'utilisateur sont reçues par le point d'entrée de l'application puis orientées vers le contrôleur correspondant à l'action demandée.

Les contrôleurs assurent le traitement des requêtes et utilisent les modèles lorsque des données doivent être récupérées, ajoutées ou modifiées. Les modèles centralisent les échanges avec les bases de données. Les informations obtenues sont ensuite transmises aux vues chargées de générer l'interface présentée à l'utilisateur.

Cette séparation permet de limiter le mélange entre la logique de présentation, la logique applicative et l'accès aux données. Elle facilite également la maintenance et l'évolution des différentes fonctionnalités du projet.

1.2 Organisation MVC

L'application est structurée selon le modèle MVC afin de répartir le code selon différentes responsabilités. Les **modèles** regroupent les opérations liées aux données. Ils permettent notamment de récupérer, ajouter, modifier ou supprimer les informations nécessaires au fonctionnement de l'application en communiquant avec les bases de données.

Les **contrôleurs** assurent le traitement des actions demandées par l'utilisateur. Ils vérifient les informations reçues, appliquent les traitements nécessaires, font appel aux modèles et transmettent les données aux vues correspondantes.

Les **vues** sont responsables de l'affichage des pages de l'application. Elles utilisent les données fournies par les contrôleurs pour générer le contenu HTML présenté à l'utilisateur.

Le fichier public/index.php constitue le point d'entrée principal de l'application. Il permet d'orienter les différentes requêtes vers les traitements correspondants.

Cette organisation facilite la séparation des responsabilités et permet de faire évoluer une partie de l'application sans regrouper l'ensemble de la logique dans les mêmes fichiers.

En complément de cette organisation MVC, le dossier Entities contient les entités utilisées pour représenter certaines données métier sous forme d'objets, tandis que le dossier Helpers regroupe des traitements utilitaires réutilisables dans différentes parties de l'application.

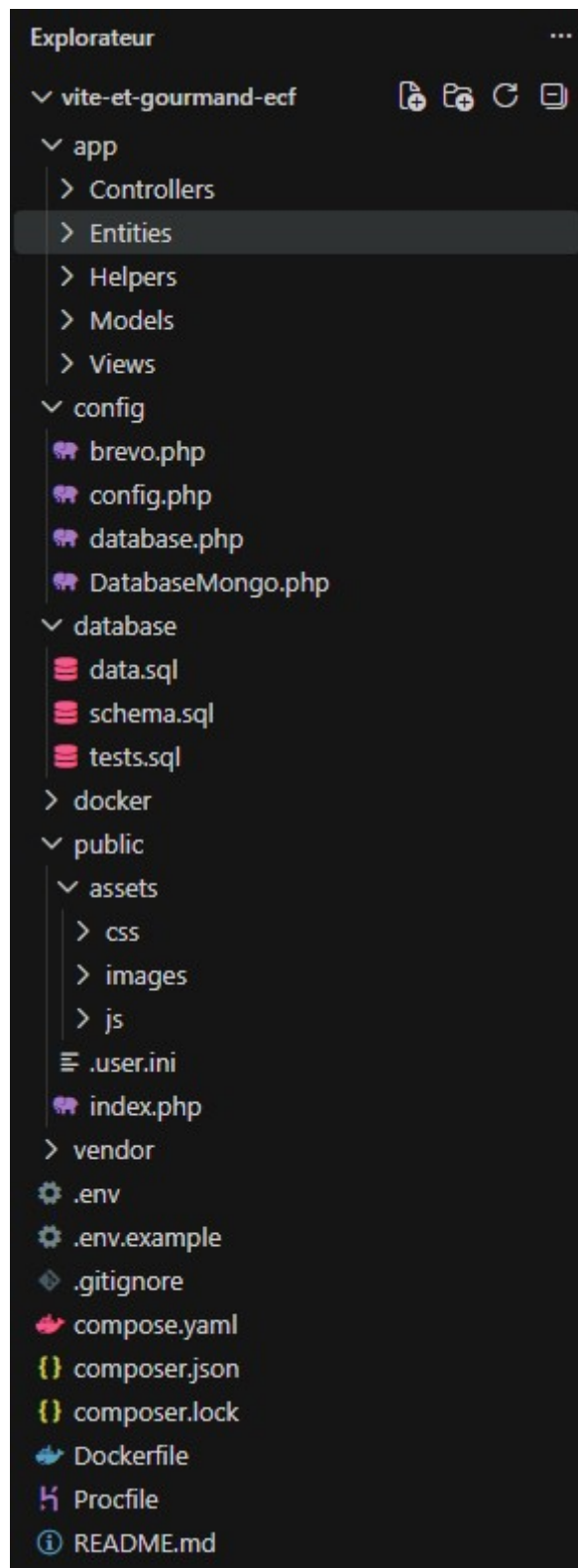


Figure 1: Arborescence et organisation technique du projet Vite & Gourmand

1.3 Technologies utilisées

Le développement de l'application Vite & Gourmand repose sur plusieurs technologies et services complémentaires, utilisés aussi bien pour le développement de l'application que pour la gestion des données, l'envoi des e-mails et son déploiement.

PHP 8.3 est utilisé côté serveur pour gérer la logique de l'application, le traitement des requêtes et des formulaires, l'authentification, les commandes ainsi que les différentes fonctionnalités accessibles selon les rôles des utilisateurs. L'application s'exécute avec **Apache** comme serveur web dans l'environnement conteneurisé.

MySQL constitue la base de données relationnelle principale. Elle permet de stocker notamment les utilisateurs, les menus, les plats, les commandes, les avis et les différentes informations nécessaires au fonctionnement de l'application.

MongoDB est utilisé comme base de données NoSQL pour la partie non relationnelle du projet. Son utilisation et les données concernées sont détaillées dans la partie consacrée à la gestion des données.

L'interface utilisateur est développée avec **HTML, CSS et JavaScript**. **Bootstrap 5.3.3** est utilisé pour certains composants et pour faciliter l'adaptation de l'interface aux différentes tailles d'écran. **Bootstrap Icons** fournit différentes icônes utilisées dans l'interface. Les polices **Montserrat, Playfair Display et Raleway** sont intégrées à l'aide de **Google Fonts**.

Composer permet de gérer les dépendances PHP nécessaires au projet. Parmi celles-ci figurent notamment le SDK PHP de **Brevo**, utilisé pour l'envoi des e-mails transactionnels de l'application, ainsi que **Guzzle**, utilisé pour les échanges HTTP nécessaires aux bibliothèques du projet.

Docker et **Docker Compose** sont utilisés pour conteneuriser l'environnement de développement et regrouper les différents services nécessaires à l'exécution de l'application, notamment PHP avec Apache, MySQL et MongoDB. Cette configuration permet de reproduire plus facilement l'environnement nécessaire au fonctionnement du projet.

Git et **GitHub** sont utilisés pour le versionnement du code source. L'utilisation de branches et de Pull Requests permet d'isoler les développements avant leur intégration dans les branches principales du projet.

Name.com est utilisé pour l'enregistrement et la gestion du nom de domaine associé au projet ainsi que pour sa configuration DNS.

Brevo assure l'envoi des e-mails transactionnels générés par l'application, notamment les messages liés aux comptes utilisateurs et au suivi de certaines actions effectuées dans l'application.

Heroku est utilisé pour l'hébergement et le déploiement de la version en ligne de l'application. La configuration de l'environnement de production est séparée de celle utilisée pour le développement local.

Enfin, les paramètres sensibles ou dépendants de l'environnement, tels que les identifiants de connexion aux bases de données, les clés d'API et les paramètres des services externes, sont gérés à l'aide de **variables d'environnement** afin de ne pas intégrer ces informations directement dans le code source.

2 Conception et gestion des données

La gestion des données constitue une partie importante de l'application Vite & Gourmand. Le projet utilise principalement une base de données relationnelle MySQL pour stocker les informations nécessaires au fonctionnement de l'application, complétée par MongoDB pour certaines données utilisées dans la partie statistique.

La conception de la base relationnelle a été réalisée à partir d'un modèle conceptuel de données afin d'identifier les principales entités du projet, leurs informations et les relations existantes entre elles.

Cette organisation permet notamment de gérer les utilisateurs et leurs rôles, les menus et les plats proposés, les commandes et leur suivi, ainsi que les avis déposés par les utilisateurs.

2.1 Modèle conceptuel de données

Avant la création de la base de données MySQL, un modèle conceptuel de données a été réalisé afin de représenter les différentes données nécessaires au fonctionnement de l'application et les relations existantes entre elles.

Le modèle a été construit à partir des principales fonctionnalités définies dans le cahier des charges. La gestion des comptes utilisateurs nécessite par exemple de conserver les informations des utilisateurs et de leur associer un rôle. La gestion des commandes nécessite quant à elle de relier une commande à un utilisateur et à un menu, tout en conservant les informations nécessaires à la livraison et au suivi de son état.

Les menus sont séparés des plats afin qu'un même plat puisse être associé à différents menus. Les allergènes sont également gérés séparément et associés aux plats concernés. Cette organisation évite de répéter inutilement les mêmes informations et facilite leur modification.

Le suivi d'une commande a été séparé de la commande elle-même. Une commande possède un statut courant, mais son évolution doit également pouvoir être retracée. Les changements de statut sont donc conservés afin de constituer un historique du suivi de la commande.

Les avis sont associés à l'utilisateur qui les rédige ainsi qu'à la commande concernée. Cette organisation permet de vérifier qu'un avis correspond bien à une prestation réalisée et de conserver le lien avec l'utilisateur ayant déposé cet avis. Un état de validation est également conservé afin de permettre la modération des avis avant leur affichage.

Enfin, les informations utilisées pour le calcul de la livraison sont organisées de manière à permettre l'application des règles métier prévues, notamment la distinction entre une livraison à Bordeaux et une livraison en dehors de Bordeaux selon la distance.

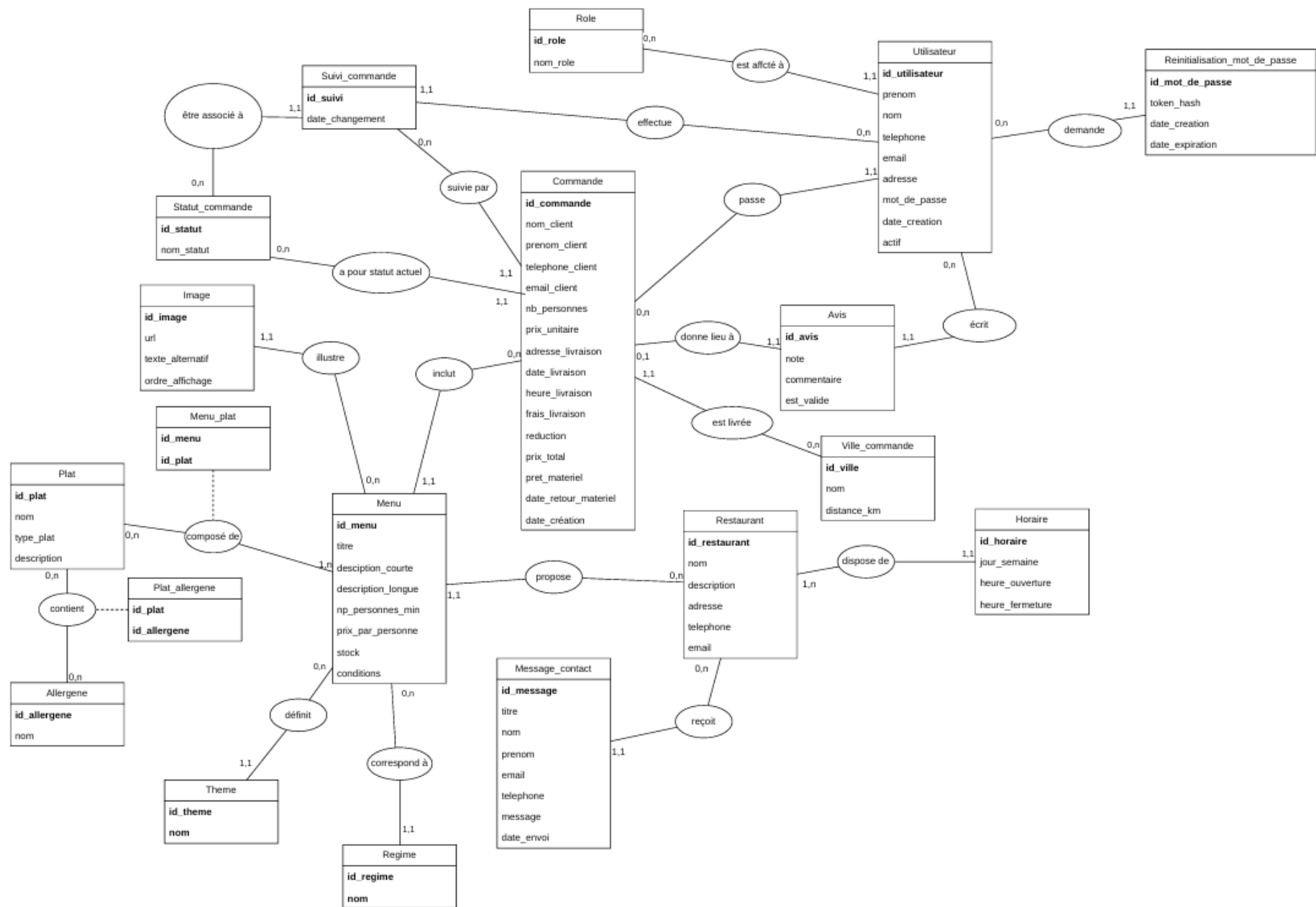


Figure 2: Capture du MCD

2.2 Base de données relationnelle MySQL

La majorité des données nécessaires au fonctionnement de l'application est stockée dans une base de données relationnelle MySQL. La structure de cette base a été réalisée à partir du modèle conceptuel de données présenté précédemment.

Chaque entité principale du modèle est représentée par une table contenant une clé primaire permettant d'identifier de manière unique chaque enregistrement. Les relations entre les données sont assurées par des clés étrangères.

La table **utilisateur** contient les informations nécessaires à la gestion des comptes. Chaque utilisateur est associé à un rôle permettant de distinguer les clients, les employés et les administrateurs. L'adresse e-mail est unique afin d'éviter la création de plusieurs comptes avec la même adresse.

Les informations relatives aux prestations sont réparties entre plusieurs tables. La table **menu** contient les caractéristiques générales d'un menu, tandis que les plats sont stockés séparément dans la table **plat**. Une table associative permet de créer la relation entre les menus et les plats. Le même principe est utilisé entre les plats et leurs allergènes afin d'éviter la duplication des informations.

La table **commande** centralise les informations nécessaires à une commande : client, menu sélectionné, nombre de personnes, prix unitaire, adresse et informations de livraison ainsi que les données liées au prêt éventuel de matériel. Elle est reliée aux autres données nécessaires au traitement de la commande par des clés étrangères.

La gestion de l'évolution des commandes repose également sur les tables **statut_commande** et **suivi_commande**. Le statut courant permet de connaître l'état actuel d'une commande, tandis que le suivi conserve les différents changements intervenus au cours de son traitement. D'autres tables permettent de gérer les fonctionnalités complémentaires de l'application, notamment les avis clients, les horaires du restaurant, les messages provenant du formulaire de contact et les demandes de réinitialisation de mot de passe.

L'utilisation d'une base relationnelle permet ainsi de structurer les données, de limiter leur duplication et de maintenir les relations nécessaires entre les différentes fonctionnalités de l'application.

Table	Action	Lignes	Type	Interclassement	Taille	Perte
☐ allergene	★	14	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ avis	★	0	InnoDB	utf8mb4_unicode_ci	48,0 kio	-
☐ commande	★	0	InnoDB	utf8mb4_unicode_ci	80,0 kio	-
☐ horaire	★	7	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ image	★	18	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ menu	★	6	InnoDB	utf8mb4_unicode_ci	64,0 kio	-
☐ menu_plat	★	18	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ message_contact	★	0	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ plat	★	18	InnoDB	utf8mb4_unicode_ci	16,0 kio	-
☐ plat_allergene	★	32	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ regime	★	3	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ reinitialisation_mot_de_passe	★	0	InnoDB	utf8mb4_unicode_ci	48,0 kio	-
☐ restaurant	★	1	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ role	★	3	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ statut_commande	★	8	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ suivi_commande	★	0	InnoDB	utf8mb4_unicode_ci	64,0 kio	-
☐ theme	★	4	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
☐ utilisateur	★	1	InnoDB	utf8mb4_unicode_ci	48,0 kio	-
☐ ville_commande	★	8	InnoDB	utf8mb4_unicode_ci	32,0 kio	-
19 tables	Somme	141	MyISAM	utf8mb4_unicode_ci	752,0 kio	0 0

Figure 3: Structure de la base de données relationnelle MySQL dans phpMyAdmin

2.3 Accès aux données avec PDO

L'accès à la base de données MySQL est réalisé en PHP à l'aide de **PDO (PHP Data Objects)**. La création de la connexion est centralisée dans une classe Database afin que les différents modèles de l'application utilisent le même mécanisme d'accès aux données.

Cette classe utilise un fonctionnement inspiré du **patron de conception Singleton**. Une instance de PDO est conservée dans une propriété statique et n'est créée que lors du premier appel à la méthode getConnection(). Les appels suivants réutilisent cette même instance au lieu de créer systématiquement une nouvelle connexion.

La configuration de la connexion dépend de l'environnement dans lequel l'application est exécutée. En environnement local ou avec Docker, les paramètres peuvent être récupérés à partir des variables d'environnement ou du fichier .env. En production sur Heroku, la connexion peut être construite à partir de la variable JAWSDB_URL fournie par le service de base de données. Cette organisation permet de conserver le même code tout en utilisant des paramètres de connexion différents selon l'environnement d'exécution. La connexion utilise l'encodage utf8mb4. PDO est également configuré pour déclencher des exceptions en cas d'erreur, retourner par défaut les résultats sous forme de tableaux associatifs et utiliser les requêtes préparées natives du serveur MySQL.

Dans l'architecture MVC de l'application, les modèles récupèrent cette connexion à l'aide de Database::getConnection() puis exécutent les requêtes nécessaires à leurs fonctionnalités. Les valeurs variables sont transmises à l'aide de requêtes préparées afin de ne pas les intégrer directement dans les instructions SQL.

Par exemple, lors de la recherche d'un utilisateur à partir de son adresse e-mail, la requête SQL utilise un paramètre qui est ensuite transmis séparément lors de son exécution. Ce mécanisme est également utilisé dans les autres modèles manipulant les commandes, les menus, les avis et les comptes utilisateurs.

L'utilisation des requêtes préparées contribue notamment à limiter les risques d'injection SQL.

Cette organisation permet ainsi de centraliser la configuration de la connexion, de séparer l'accès aux données du reste de la logique de l'application et de conserver un fonctionnement adapté aux différents environnements de développement et de production.

```
<?php

class Database
{
    private static ?PDO $pdo = null;

    public static function getConnection(): PDO
    {
        if (self::$pdo === null) {
            // Configuration de la connexion...
            self::$pdo = new PDO($dsn, $user, $password, [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES => false
            ]);
        }

        return self::$pdo;
    }
}
```

Figure 4: Début et fin de la centralisation et réutilisation de la connexion PDO

```

class UserModel
{
    public function findByEmail(string $email): ?array
    {
        $pdo = Database::getConnection();

        $sql = "
            SELECT
                utilisateur.id,
                utilisateur.nom,
                utilisateur.prenom,
                utilisateur.email,
                utilisateur.telephone,
                utilisateur.adresse,
                utilisateur.mot_de_passe_hash,
                utilisateur.actif,
                role.nom AS role
            FROM utilisateur
            INNER JOIN role ON utilisateur.role_id = role.id
            WHERE utilisateur.email = ?
        ";

        $stmt = $pdo->prepare($sql);
        $stmt->execute([$email]);

        $user = $stmt->fetch();

        return $user ?: null;
    }
}

```

Figure 5: Exemple de requête préparée avec PDO dans UserModel

2.4 Base de données NoSQL MongoDB

En complément de la base de données relationnelle MySQL, l'application utilise MongoDB pour la gestion et l'exploitation des données statistiques accessibles depuis l'espace administrateur. MySQL reste la source principale des données métier de l'application. Les informations nécessaires aux statistiques sont d'abord calculées ou extraites à partir des commandes enregistrées dans MySQL, puis elles sont synchronisées vers MongoDB. Les pages de statistiques utilisent ensuite les données présentes dans MongoDB pour réaliser leur affichage.

Deux collections sont principalement utilisées dans le modèle StatsModel.

La collection statistiques_menu contient des données agrégées pour chaque menu, notamment son identifiant, son nom, le nombre de commandes terminées, le chiffre d'affaires généré, le prix moyen ainsi que la date de synchronisation. Ces données permettent notamment d'afficher la comparaison du nombre de commandes terminées par menu.

La collection chiffre_affaires_commandes conserve les données nécessaires à l'analyse du chiffre d'affaires des commandes terminées. Chaque document contient notamment l'identifiant de la commande, l'identifiant et le nom du menu, la date de fin de la commande et le chiffre d'affaires correspondant. Ces données peuvent être filtrées selon un menu, une date de début et une date de fin.

À partir de ces informations, l'espace administrateur peut notamment présenter le chiffre d'affaires total, le nombre de commandes terminées, le menu le plus rentable, ainsi qu'un détail du chiffre d'affaires et du prix moyen par menu.

La connexion à MongoDB est centralisée dans la classe DatabaseMongo. Celle-ci conserve une instance du client MongoDB et récupère sa configuration à partir des variables d'environnement. L'application peut utiliser une URI complète avec MONGO_URI ou construire la connexion à partir de MONGO_HOST et MONGO_PORT. La base utilisée est définie avec MONGO_DATABASE. Le fonctionnement mis en place repose ainsi sur une synchronisation MySQL vers MongoDB : MySQL conserve les données métier de référence, tandis que MongoDB est utilisé pour la partie statistique et analytique de l'application.

```
<?php

require_once __DIR__ . '/../vendor/autoload.php';

use MongoDB\Client;
use MongoDB\Database;

class DatabaseMongo
{
    private static ?Client $client = null;

    public static function getConnection(): Client
    {
        if (self::$client === null) {
            $mongoUri = getenv('MONGO_URI');

            if ($mongoUri) {
                self::$client = new Client($mongoUri);
            } else {
                $host = getenv('MONGO_HOST') ?: 'localhost';
                $port = getenv('MONGO_PORT') ?: '27017';

                self::$client = new Client(
                    "mongodb://{ $host }: { $port }"
                );
            }
        }

        return self::$client;
    }

    public static function getDatabase(): Database
    {
        $database = getenv('MONGO_DATABASE')
            ?: 'vite_et_gourmand_stats';

        return self::getConnection()->selectDatabase($database);
    }
}
```

Figure 6: Centralisation de la connexion à MongoDB dans DatabaseMongo

```

public function statistics(): void
{
    Auth::requireRole(['Admin']);

    $statsSqlModel = new StatsSqlModel();
    $statsMongoModel = new StatsModel();

    // 1. Calcul depuis MySQL
    $statisticsFromSql = $statsSqlModel->getStatisticsByMenu();

    // 2. Synchronisation vers MongoDB
    $statsMongoModel->synchronizeFromSql($statisticsFromSql);

    // 3. Lecture depuis MongoDB
    $statistics = $statsMongoModel->getAll();

    // 4. Total des commandes pour les pourcentages
    $totalCommandes = array_sum(
        array_column($statistics, 'commandes')
    );

    require_once __DIR__ . '/../Views/pages/admin-statistics.php';
}

```

Figure 7: Synchronisation des statistiques MySQL vers MongoDB

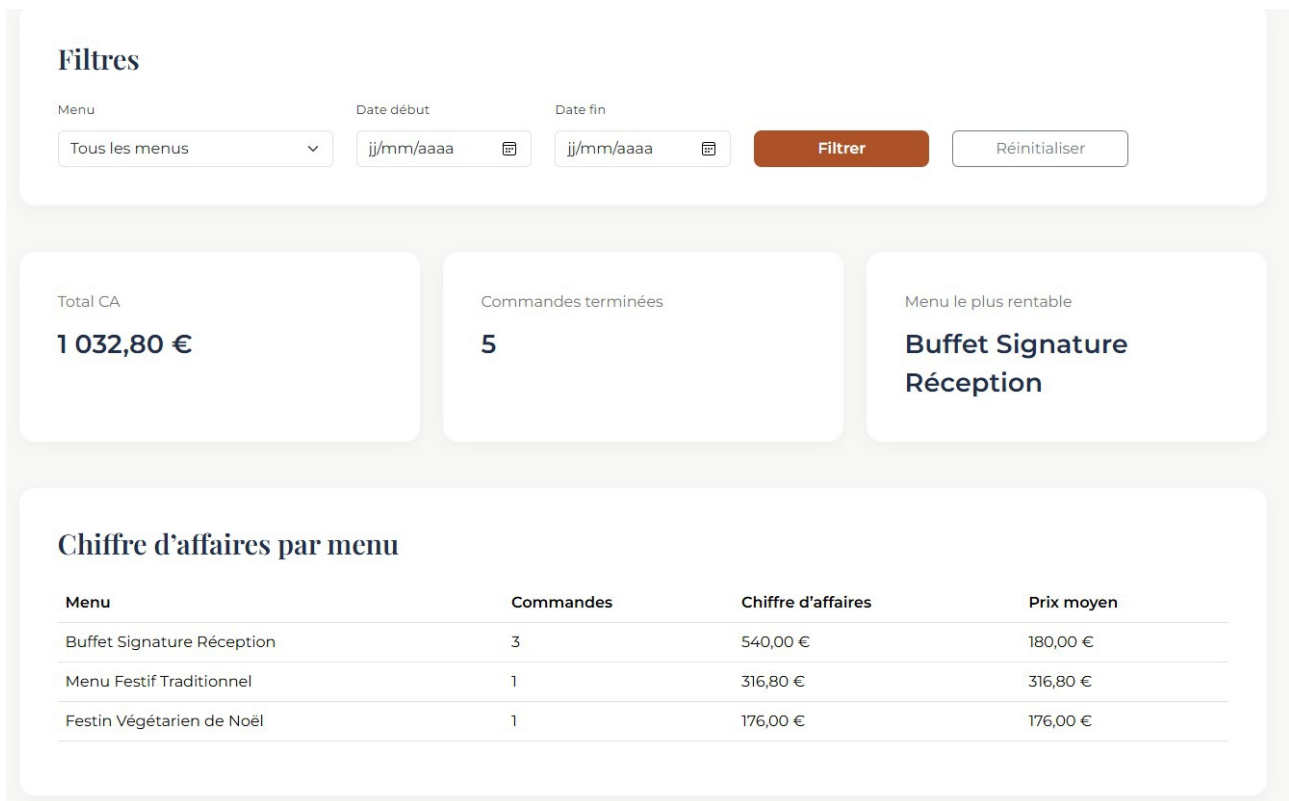


Figure 8: Exploitation des données statistiques MongoDB dans l'espace administrateur

2.5 Initialisation et jeu de données

Afin de faciliter l'installation du projet sur un nouvel environnement, les fichiers nécessaires à la création et à l'initialisation de la base de données sont intégrés au dépôt.

Le fichier database/schema.sql contient la structure de la base de données relationnelle MySQL. Il permet de créer les différentes tables, leurs colonnes, les clés primaires et étrangères ainsi que les contraintes nécessaires au fonctionnement de l'application.

Le fichier database/data.sql contient le jeu de données initial utilisé par l'application. Il permet notamment d'insérer les rôles, les informations du restaurant, les horaires, les thèmes, les régimes alimentaires, les allergènes, les plats, les menus ainsi que les différentes données nécessaires au démarrage du projet.

L'initialisation de MySQL est automatisée dans la configuration Docker. Les fichiers schema.sql et data.sql sont montés dans le répertoire /docker-entrypoint-initdb.d/ du conteneur MySQL. Ils sont préfixés respectivement par 01- et 02- afin que la création de la structure soit exécutée avant l'insertion des données.

Les données MySQL sont conservées dans un volume Docker persistant nommé mysql_data. MongoDB dispose également de son propre volume mongodb_data. Les données ne sont donc pas perdues lors d'un simple arrêt ou redémarrage des conteneurs.

```
mysql:
  image: mysql:8.4
  container_name: vite_et_gourmand_mysql
  restart: unless-stopped
  environment:
    MYSQL_ROOT_PASSWORD: "${MYSQL_ROOT_PASSWORD}"
    MYSQL_DATABASE: "${DB_NAME}"
    MYSQL_USER: "${DB_USER}"
    MYSQL_PASSWORD: "${DB_PASSWORD}"
  ports:
    - "3305:3306"
  volumes:
    - mysql_data:/var/lib/mysql
    - ./database/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro
    - ./database/data.sql:/docker-entrypoint-initdb.d/02-data.sql:ro
```

Figure 9: Initialisation automatique de MySQL avec les scripts schema.sql et data.sql

Les dépendances PHP sont également installées automatiquement au démarrage du conteneur PHP. La commande définie dans compose.yaml exécute composer install --no-interaction --prefer-dist avant le lancement du serveur Apache.

Après récupération du projet et configuration du fichier .env, l'environnement Docker peut ainsi être construit et démarré avec la commande : docker compose up -d --build

Cette commande permet de démarrer les trois services nécessaires au projet : PHP avec Apache, MySQL et MongoDB. Les variables définies dans le fichier .env sont utilisées pour transmettre aux conteneurs les paramètres de connexion nécessaires sans les inscrire directement dans le code source.

```
[+] up 5/5
✓ Image vite-et-gourmand-ecf-php      Built
✓ Network vite-et-gourmand-ecf_default Created
✓ Container vite_et_gourmand_mongodb  Started
✓ Container vite_et_gourmand_mysql    Started
✓ Container vite_et_gourmand_php       Started
```

Figure 10: Construction et démarrage des services PHP, MySQL et MongoDB avec Docker Compose

L'application peut également être exécutée localement sans Docker à l'aide de WAMP. WAMP fournit Apache, PHP et MySQL, mais pas le serveur MongoDB. Pour utiliser les fonctionnalités statistiques en

environnement local sans Docker, MongoDB doit donc être installé et démarré séparément. La connexion entre l'application et MongoDB est ensuite gérée par la classe DatabaseMongo.php à partir des variables d'environnement définies pour le projet.

La base de données MySQL peut être créée manuellement en important successivement les fichiers schema.sql et data.sql depuis phpMyAdmin. Les paramètres de connexion sont alors adaptés dans le fichier .env à l'environnement local.

Cette double possibilité permet d'utiliser un environnement local classique pendant le développement, tout en proposant avec Docker une installation plus facilement reproductible regroupant les différents services nécessaires au fonctionnement complet de l'application.

3 Architecture et fonctionnement de l'application

L'application Vite & Gourmand est organisée selon une architecture **MVC (Modèle-Vue-Contrôleur)**. Cette séparation permet de répartir les responsabilités entre l'accès aux données, le traitement des actions de l'utilisateur et l'affichage des pages.

Cette organisation est utilisée pour les différentes fonctionnalités de l'application, aussi bien dans l'espace client que dans les espaces réservés aux employés et aux administrateurs.

3.1 Architecture MVC

L'architecture de l'application repose sur trois catégories principales de composants : les modèles, les vues et les contrôleurs.

Les modèles sont responsables de l'accès et de la manipulation des données. Ils communiquent notamment avec MySQL à l'aide de PDO ou, pour les fonctionnalités statistiques, avec MongoDB. Ils contiennent les méthodes permettant de récupérer, ajouter, modifier ou supprimer les données nécessaires au fonctionnement de l'application.

Les contrôleurs assurent le lien entre les actions réalisées par l'utilisateur, les modèles et les vues. Ils récupèrent les données nécessaires auprès des modèles, appliquent les traitements liés à la fonctionnalité demandée puis transmettent les informations aux vues. Ils interviennent également dans le contrôle des droits d'accès à certaines fonctionnalités.

Les vues sont responsables de la présentation des informations à l'utilisateur. Elles contiennent principalement la structure HTML ainsi que le PHP nécessaire à l'affichage des données fournies par les contrôleurs. La mise en forme est ensuite assurée par les feuilles de style CSS et les composants Bootstrap utilisés dans le projet.

Lorsqu'un utilisateur accède à une fonctionnalité, la requête est dirigée vers le contrôleur correspondant. Celui-ci peut appeler un ou plusieurs modèles afin de récupérer ou de modifier des données. Une fois le traitement terminé, le contrôleur charge la vue correspondante avec les informations nécessaires à son affichage.

Cette séparation permet de limiter le mélange entre la logique d'accès aux données, le traitement des fonctionnalités et leur présentation. Elle facilite ainsi l'organisation, la maintenance et l'évolution du code.

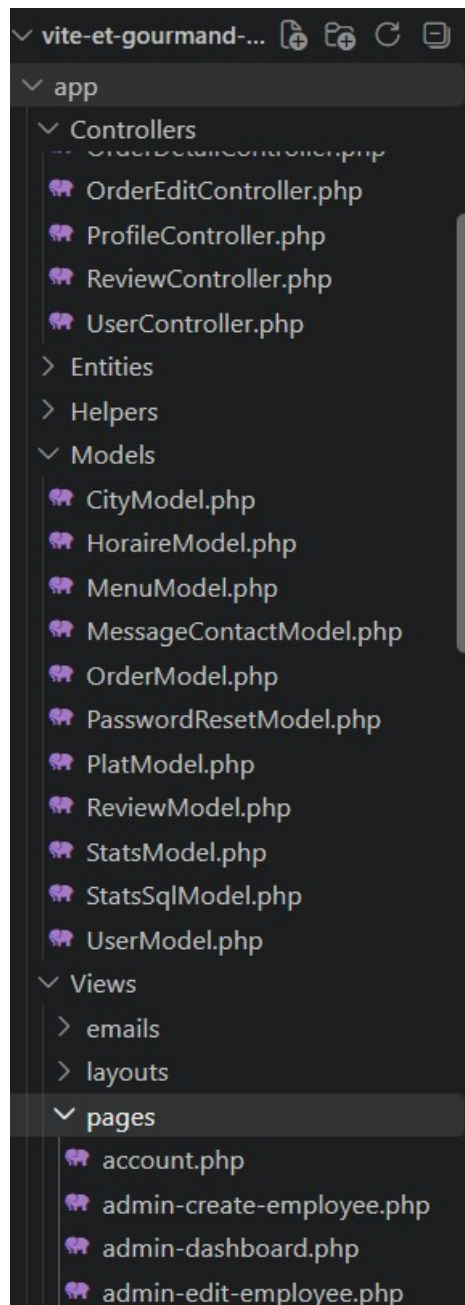


Figure 11: Organisation des modèles, vues et contrôleurs dans l'architecture MVC

3.2 Utilisation des entités

En complément de l'architecture MVC, certaines règles métier de l'application ont été regroupées dans des classes entités. Trois entités principales ont été créées : **Commande**, **Menu** et **Utilisateur**.

Ces classes permettent de représenter des éléments métier importants de l'application sous forme d'objets possédant leurs propres données et comportements. Elles évitent ainsi de placer certaines règles métier directement dans les contrôleurs ou les vues et permettent de les centraliser dans des classes dédiées.

L'entité **Commande** regroupe les règles nécessaires au calcul du montant d'une commande. Elle permet de calculer le sous-total à partir du nombre de personnes et du prix unitaire, d'appliquer une réduction de 10 % lorsque le nombre de personnes atteint au moins cinq personnes de plus que le minimum du menu, et de calculer les frais de livraison. Ceux-ci comprennent un montant fixe de 5 €, auquel s'ajoutent 0,59 € par kilomètre lorsque la livraison est réalisée en dehors de Bordeaux. Le montant total est ensuite obtenu à

partir de ces différents calculs. L'entité **Menu** regroupe plusieurs règles permettant de déterminer si un menu peut être commandé. Elle vérifie notamment que le nombre de personnes respecte le minimum prévu et que le stock disponible est suffisant. Elle permet également de déterminer le seuil à partir duquel la réduction d'une commande peut s'appliquer et de vérifier la validité des principales valeurs d'un menu. L'entité **Utilisateur** centralise plusieurs règles de validation et de contrôle liées aux comptes utilisateurs. Elle permet notamment de vérifier qu'un utilisateur est actif, de contrôler son rôle, de valider le format de son adresse e-mail ainsi que certaines longueurs de données. Des méthodes statiques sont également utilisées pour vérifier le format d'un numéro de téléphone et le respect des règles définies pour les mots de passe.

Les propriétés de ces entités sont déclarées en visibilité private et sont initialisées à l'aide de leurs constructeurs. Les interactions avec leurs données passent ensuite par les méthodes prévues par les classes. Cette organisation met notamment en œuvre le principe d'encapsulation de la programmation orientée objet.

L'utilisation de ces entités a été ciblée sur les éléments pour lesquels des règles métier spécifiques devaient être regroupées. Les différentes tables de la base de données ne possèdent donc pas systématiquement une entité PHP correspondante : les modèles restent responsables de l'accès et de la persistance des données, tandis que les entités **Commande**, **Menu** et **Utilisateur** portent certaines règles métier et validations associées à ces éléments.

```

class Commande
{
    public function __construct()
    {

    }

    public function calculerSousTotal(): float
    {
        return $this->prixUnitaire * $this->nbPersonnes;
    }

    public function calculerReduction(): float
    {
        if ($this->nbPersonnes >= $this->minimumMenu + 5) {
            return $this->calculerSousTotal() * 0.10;
        }

        return 0;
    }

    public function calculerFraisLivraison(): float
    {
        $fraisLivraison = 5;

        if ($this->distanceKm > 0) {
            $fraisLivraison += $this->distanceKm * 0.59;
        }

        return $fraisLivraison;
    }

    public function calculerTotal(): float
    {
        return $this->calculerSousTotal()
            - $this->calculerReduction()
            + $this->calculerFraisLivraison();
    }
}

```

Figure 12: Exemple d'encapsulation des règles métier dans l'entité Commande

3.3 Routage et contrôleurs

Le point d'entrée principal de l'application est le fichier public/index.php. Il centralise le routage des différentes requêtes et permet de diriger l'utilisateur vers le contrôleur correspondant à la fonctionnalité demandée.

La route à traiter est récupérée à partir du paramètre URL présent dans l'URL. Lorsqu'aucune valeur n'est fournie, la route accueil est utilisée par défaut :

```
$url = $_GET['url'] ?? 'accueil';
```

Le routage repose ensuite sur une structure switch. Chaque route est associée à un contrôleur et à l'une de ses méthodes. Par exemple, la route menus instancie MenuController et appelle sa méthode index(), tandis que la route commande utilise la méthode create() de OrderController. Les contrôleurs sont organisés selon les différentes fonctionnalités de l'application. Des contrôleurs spécifiques prennent notamment en charge l'authentification, les menus, les commandes, le profil utilisateur, les avis ainsi que les espaces employé et administrateur. Certaines fonctionnalités d'un même espace sont regroupées dans un contrôleur commun. EmployeeController prend par exemple en charge le tableau de bord employé, la gestion des commandes, des menus, des plats et des horaires. AdminController gère notamment le tableau de bord administrateur, les employés et les statistiques.

Les contrôleurs peuvent ensuite faire appel aux modèles pour récupérer ou modifier les données nécessaires, effectuer les traitements prévus, puis charger la vue correspondant au résultat de l'action. Le contrôle des autorisations pour les fonctionnalités protégées est réalisé à l'aide du helper Auth, notamment en fonction du rôle de l'utilisateur connecté.

Une route spécifique, api-menus, est également utilisée pour accéder à la méthode apiSearch() de MenuController. Elle permet de fournir les données nécessaires à la recherche dynamique des menus côté interface.

Enfin, lorsqu'aucune route ne correspond à la valeur demandée, l'application retourne un code HTTP 404 et indique que la page est introuvable.

Cette organisation fournit ainsi un système de routage centralisé et adapté à la taille du projet, tout en conservant la séparation entre le point d'entrée de l'application, les contrôleurs, les modèles et les vues.

```
$url = $_GET['url'] ?? 'accueil';

switch ($url) {

    case 'accueil':
        $controller = new HomeController();
        $controller->index();
        break;
    case 'menus':
        $controller = new MenuController();
        $controller->index();
        break;
    case 'menu-detail':
        $controller = new MenuDetailController();
        $controller->index();
        break;
    case 'connexion':
        $controller = new AuthController();
        $controller->login();
        break;
    case 'deconnexion':
        $controller = new AuthController();
        $controller->logout();
        break;
```

Figure 13: Routage des requêtes vers les contrôleurs dans
public/index.php

3.4 Authentification et gestion des rôles

L'authentification de l'application repose sur les sessions PHP, la vérification sécurisée des mots de passe et un contrôle des droits d'accès selon le rôle de l'utilisateur connecté.

Lors de la connexion, l'adresse e-mail et le mot de passe saisis sont d'abord contrôlés. L'utilisateur correspondant est recherché dans la base de données, puis le mot de passe est vérifié avec `password_verify()` à partir du hash enregistré. En cas de succès, l'identifiant de session est régénéré avec `session_regenerate_id(true)` avant l'enregistrement des informations utiles dans `$_SESSION['user']`.

L'entité Utilisateur est utilisée pour vérifier que le compte est actif et pour contrôler son rôle. Après authentification, l'utilisateur est redirigé vers l'espace correspondant : espace administrateur, espace employé ou espace client.

L'inscription applique plusieurs contrôles avant la création du compte : présence des champs obligatoires, validation de l'adresse e-mail, validation du numéro de téléphone, contrôle du mot de passe, vérification de l'unicité de l'adresse e-mail et contrôle de la longueur de certaines données. Le mot de passe est ensuite enregistré sous forme de hash à l'aide de `password_hash()` avec `PASSWORD_DEFAULT`.

La gestion des accès protégés est centralisée dans le helper Auth. La méthode `requireRole()` vérifie qu'un utilisateur est connecté, recharge ses informations depuis la base de données, s'assure que son compte est toujours actif puis vérifie qu'il possède l'un des rôles autorisés pour la page demandée. En cas d'échec, l'utilisateur est redirigé vers la page de connexion ou vers l'accueil.

Un délai d'inactivité de 30 minutes est également appliqué aux sessions protégées. Lorsqu'il est dépassé, la session est détruite et l'utilisateur est redirigé vers la page de connexion. Cette vérification permet de limiter la durée d'exposition d'une session laissée inactive.

La déconnexion supprime les données de session puis détruit la session courante avant de rediriger l'utilisateur vers l'accueil.

L'application propose également une fonctionnalité de réinitialisation du mot de passe. Lorsqu'un utilisateur en fait la demande, un jeton aléatoire est généré avec `random_bytes()`. Seul son hash SHA-256 est conservé dans la base de données, tandis que le jeton original est envoyé dans le lien de réinitialisation par e-mail. Ce lien possède une durée de validité limitée à une heure.

Lors de la réinitialisation, le jeton reçu est de nouveau haché puis comparé au hash enregistré. Si le jeton est valide et non expiré, le nouveau mot de passe est contrôlé puis enregistré sous forme de hash. Le jeton de réinitialisation est ensuite supprimé afin d'empêcher sa réutilisation.

```

class Auth

    public static function requireRole(array $roles): void
    {
        self::startSession();
        self::checkSessionTimeout();

        if (!isset($_SESSION['user'])) {
            header('Location: index.php?url=connexion');
            exit;
        }

        $userModel = new UserModel();

        $user = $userModel->findById(
            (int) $_SESSION['user']['id']
        );

        if (!$user) {
            $_SESSION = [];
            session_destroy();

            header('Location: index.php?url=connexion');
            exit;
        }

        $utilisateur = new Utilisateur(
            $user['nom'],
            $user['prenom'],
            $user['email'],
            (bool) $user['actif'],
            $user['role']
        );

        if (!$utilisateur->estActif()) {
            $_SESSION = [];
            session_destroy();

            header('Location: index.php?url=connexion');
            exit;
        }
    }

```

Figure 15: Contrôle des rôles et de l'état du compte dans le helper Auth

3.5 Gestion des menus

La gestion des menus comprend une partie accessible aux visiteurs et aux clients ainsi qu'une partie d'administration accessible aux employés et aux administrateurs.

Sur la partie publique de l'application, **MenuController** utilise **MenuModel** pour récupérer les menus enregistrés dans MySQL et les transmettre à la vue. Chaque menu présente notamment son titre, sa description, son nombre minimum de personnes, son prix par personne, son régime alimentaire et son image principale.

Une page de détail permet d'obtenir les informations complémentaires d'un menu.

MenuDetailController récupère séparément les informations générales du menu, ses différentes images ainsi que les plats qui le composent. Les allergènes associés aux plats sont également récupérés grâce aux relations présentes dans la base de données.

La liste des menus dispose également d'un système de recherche dynamique. Une route **apimenus** appelle la méthode **apiSearch()** de **MenuController**. Les critères sélectionnés dans l'interface sont transmis au serveur et peuvent porter sur le thème, le régime alimentaire, le prix minimum, le prix maximum et le nombre de personnes. **MenuModel::searchMenus()** construit ensuite la requête SQL en fonction des critères renseignés et retourne les résultats au format JSON.

La gestion des menus est accessible aux utilisateurs possédant le rôle Employé ou Admin. Le contrôle est effectué avec **Auth::requireRole(['Employé', 'Admin'])**.

Depuis cet espace, un utilisateur autorisé peut créer un menu en renseignant ses différentes caractéristiques : titre, descriptions, nombre minimum de personnes, prix par personne, stock, conditions, thème et régime alimentaire. Un menu est également composé d'une entrée, d'un plat principal et d'un dessert sélectionnés parmi les plats existants.

L'entité **Menu** est utilisée lors de la création et de la modification afin de vérifier la validité de plusieurs données métier, notamment le titre, le nombre minimum de personnes, le prix et le stock. Le contrôleur complète ces vérifications en s'assurant que les autres informations nécessaires sont présentes.

Chaque menu peut également posséder entre une et trois images lors de sa création. Les fichiers sont contrôlés avant leur enregistrement et seuls les formats JPEG, PNG et WebP sont acceptés. Les images sont enregistrées dans un répertoire associé au thème et au menu, tandis que leur chemin et leur ordre d'affichage sont enregistrés dans MySQL.

Lors de la modification d'un menu, les données existantes sont récupérées afin de préremplir le formulaire. Les plats associés peuvent être remplacés et de nouvelles images peuvent être fournies. Lorsqu'aucune nouvelle image n'est envoyée, les images existantes sont conservées. Lorsqu'elles sont remplacées, les anciens fichiers et leurs références en base de données sont supprimés avant l'enregistrement des nouvelles images.

La suppression d'un menu est également contrôlée. Avant de supprimer celui-ci, l'application vérifie s'il est déjà associé à une commande. Dans ce cas, sa suppression est refusée afin de préserver la cohérence des commandes existantes.

Enfin, le stock d'un menu peut être décrémenté lors d'une commande et réincrémenté lorsque cela est nécessaire. La requête de décrémenter vérifie que la quantité disponible est suffisante avant de modifier le stock, évitant ainsi de rendre celui-ci négatif.

Filtres

Nombre de personnes

Ex : 10

Nombre de personnes prévu

Régime

Standard



Prix minimum

Ex : 20

Prix maximum

Ex : 50

Thème

Noël



Réinitialiser les filtres



Menu Festif Traditionnel

Menu festif traditionnel avec entrée, plat et dessert de saison.

Min. 5 personnes

Standard

32 € par personne

Figure 16: Recherche et filtrage dynamique des menus

```

$menuEntity = new Menu(
    $titre,
    $nbPersonnesMin,
    $prixParPersonne,
    $stock
);
if (
    !$menuEntity->aDesValeursValides() ||
    $descriptionCourte === '' ||
    $descriptionLongue === '' ||
    $conditions === '' ||
    $themeId <= 0 ||
    $regimeId <= 0 ||
    $entreeId <= 0 ||
    $platPrincipalId <= 0 ||
    $dessertId <= 0 ||
    $imageCount < 1 ||
    $imageCount > 3
) {
    $error = "Tous les champs sont obligatoires et vous devez ajouter entre 1 et 3 images.";
} else {
    $menuId = $menuModel->createMenu(
        $titre,

```

Figure 17: Validation et création d'un menu depuis l'espace employé

3.6 Gestion des commandes

La création d'une commande est accessible aux utilisateurs possédant le rôle **Client**. Le contrôleur vérifie d'abord que l'utilisateur est autorisé à accéder à la fonctionnalité, puis récupère le menu sélectionné, les villes disponibles ainsi que les informations nécessaires à l'affichage du formulaire.

Avant l'enregistrement d'une commande, plusieurs contrôles sont effectués. Le nombre de personnes doit respecter le minimum prévu pour le menu et ne pas dépasser le stock disponible. La ville sélectionnée doit être valide et la date de livraison ne peut pas être antérieure à la date du jour.

L'entité **Menu** est utilisée pour vérifier le respect du nombre minimum de personnes et la disponibilité du stock. Lorsque ces conditions sont validées, l'entité **Commande** est instanciée afin de centraliser les calculs liés au montant de la commande.

Le prix de la commande est calculé à partir du prix du menu et du nombre de personnes. Une réduction de 10 % est appliquée lorsque le nombre de personnes atteint au moins cinq personnes de plus que le minimum du menu. Les frais de livraison comprennent un forfait de 5 €, auquel s'ajoutent 0,59 € par kilomètre lorsque la distance de livraison est supérieure à zéro.

Le montant total est ainsi obtenu à partir du sous-total, de la réduction éventuelle et des frais de livraison. Ces valeurs sont ensuite enregistrées avec les autres informations de la commande, notamment les coordonnées du client, l'adresse, la date et l'heure de livraison, la ville choisie ainsi que l'éventuel prêt de matériel.

L'enregistrement complet d'une commande est réalisé dans une transaction SQL. La méthode **createCompleteOrder()** commence par créer la commande, puis décrémente le stock du menu correspondant au nombre de personnes commandées et ajoute le premier élément dans l'historique de suivi. Si l'une de ces opérations échoue, la transaction est annulée afin d'éviter de conserver une commande partiellement enregistrée.

Lorsqu'une commande est créée avec succès, un e-mail de confirmation est envoyé au client puis celui-ci est redirigé vers son espace personnel.

La modification d'une commande tient également compte du stock. Lorsque le nombre de personnes augmente, seule la différence supplémentaire est retirée du stock. Lorsqu'il diminue, la différence est réintégrée au stock. Cette opération est elle aussi réalisée dans une transaction afin de conserver la cohérence entre la commande et la disponibilité du menu.

L'annulation d'une commande réintègre les places correspondantes dans le stock du menu et ajoute le statut d'annulation à l'historique de suivi. Les opérations sont regroupées dans une transaction afin que la modification du statut, la restitution du stock et l'ajout du suivi soient appliqués ensemble ou annulés en cas d'erreur.

```
} else {  
    $commande = new Commande(  
        $nbPersonnes,  
        $menuEntity->getPrixParPersonne(),  
        $menuEntity->getNbPersonnesMin(),  
        (float) $city['distance_km']  
    );  
  
    $prixUnitaire = $menuEntity->getPrixParPersonne();  
  
    $fraisLivraison = $commande->calculerFraisLivraison();  
    $reduction = $commande->calculerReduction();  
    $prixTotal = $commande->calculerTotal();  
  
    $orderModel = new OrderModel();  
  
    $orderId = $orderModel->createCompleteOrder(  
        $nomClient,  
        $prenomClient,  
        $telephoneClient,  
        $emailClient,  
        $nbPersonnes,  
        $prixUnitaire,  
        $adresseLivraison,
```

Figure 18: Utilisation des entités Menu et Commande pour appliquer les règles métier

```

class OrderModel
{
    public function createCompleteOrder(
        int $statutId
    ): int {
        $pdo = Database::getConnection();
        try {
            $pdo->beginTransaction();

            $orderId = $this->createOrder(
                $nomClient,
                $prenomClient,
                $telephoneClient,
                $emailClient,
                $nbPersonnes,
                $prixUnitaire,
                $adresseLivraison,
                $dateLivraison,
                $heureLivraison,
                $fraisLivraison,
                $reduction,
                $prixTotal,
                $pretMateriel,
                $utilisateurId,
                $menuId,
                $villeId,
                $statutId
            );
            if ($orderId === 0) {
                throw new Exception("Impossible de créer la commande.");
            }
            $menuModel = new MenuModel();

            $stockUpdated = $menuModel->decrementStock(
                $menuId,
                $nbPersonnes
            );
            if (!$stockUpdated) {
                throw new Exception("Impossible de mettre à jour le stock.");
            }
        }
    }
}

```

Figure 19: Création d'une commande et mise à jour du stock au sein d'une transaction SQL

3.7 Suivi des commandes et gestion des statuts

Le suivi des commandes permet de contrôler leur évolution depuis leur création jusqu'à leur clôture. Chaque commande possède un statut courant, tandis que les différents changements sont enregistrés dans la table **suivi_commande** afin de conserver un historique.

Dans l'espace employé et administrateur, l'accès au détail d'une commande est protégé par **Auth::requireRole(['Employé', 'Admin'])**. La commande est récupérée à partir de son identifiant, puis l'historique des changements de statut est chargé pour être affiché dans la vue. L'évolution d'une commande suit un enchaînement contrôlé. Une commande passe successivement par les statuts **En**

attente, Acceptée, En préparation, En cours de livraison puis **Livrée**. À partir du statut livré, le traitement dépend de la présence ou non d'un prêt de matériel. Si aucun matériel n'a été prêté, la commande peut être directement terminée. Dans le cas contraire, elle passe d'abord par le statut En attente du retour du matériel, puis par Terminée après restitution.

Le contrôleur ne permet pas de sélectionner librement n'importe quel statut. Pour chaque état courant, seuls les statuts suivants autorisés sont proposés. Cette vérification est également effectuée côté serveur avant l'appel au modèle afin d'éviter un changement incohérent dans le cycle de traitement.

Lorsqu'un changement est validé, **OrderModel::updateCompleteStatus()** effectue la mise à jour du statut courant et ajoute une nouvelle entrée dans **suivi_commande**. Ces deux opérations sont regroupées dans une transaction SQL. Si l'une d'elles échoue, la transaction est annulée afin d'éviter qu'un statut soit modifié sans que son historique soit enregistré.

Chaque entrée de suivi peut également conserver l'identifiant de l'utilisateur ayant effectué l'action.

L'historique affiché dans l'espace employé/admin indique ainsi le statut, la date du changement et, lorsqu'il est disponible, le nom et le rôle de l'auteur de la modification.

Le client dispose également d'un historique de sa propre commande. L'accès est limité à ses commandes grâce à une recherche utilisant à la fois l'identifiant de la commande et celui de l'utilisateur connecté.

L'annulation d'une commande est également intégrée au suivi. Côté client, elle est autorisée uniquement lorsque la commande est encore dans son état initial prévu par l'application. Côté employé ou administrateur, l'annulation peut être réalisée sur plusieurs statuts intermédiaires, à condition que le client ait été contacté et qu'un motif soit renseigné dans le formulaire.

Lorsqu'une annulation est validée, le statut de la commande est modifié, le stock correspondant au nombre de personnes est réintégré au menu et une entrée d'annulation est ajoutée à l'historique. Ces opérations sont réalisées au sein d'une transaction afin de préserver la cohérence des données.

Le prêt de matériel introduit également une étape particulière dans le cycle de vie de la commande.

Lorsqu'une commande livrée comporte un prêt de matériel, le passage au statut En attente du retour du matériel déclenche l'envoi d'un e-mail au client concernant la restitution. Une fois le matériel retourné, la commande peut être terminée.

Lorsque le statut **Terminée** est atteint, un e-mail invitant le client à laisser un avis est également envoyé avec un lien vers le détail de sa commande.

La vue employé/admin présente enfin les informations client, les informations de commande, le détail financier, le changement de statut, l'annulation lorsque celle-ci est encore autorisée, ainsi que l'historique complet des changements. Cette page a été rendue responsive afin de rester utilisable sur ordinateur, tablette et mobile.

```
$nextStatuses = [
    1 => [2],
    2 => [3],
    3 => [4],
    4 => [5],
    6 => [7]
];

if ($currentStatus === 5) {
    $nextStatuses[5] = $hasEquipmentLoan
        ? [6]
        : [7];
}
```

Figure 20: Contrôle de l'enchaînement des statuts dans

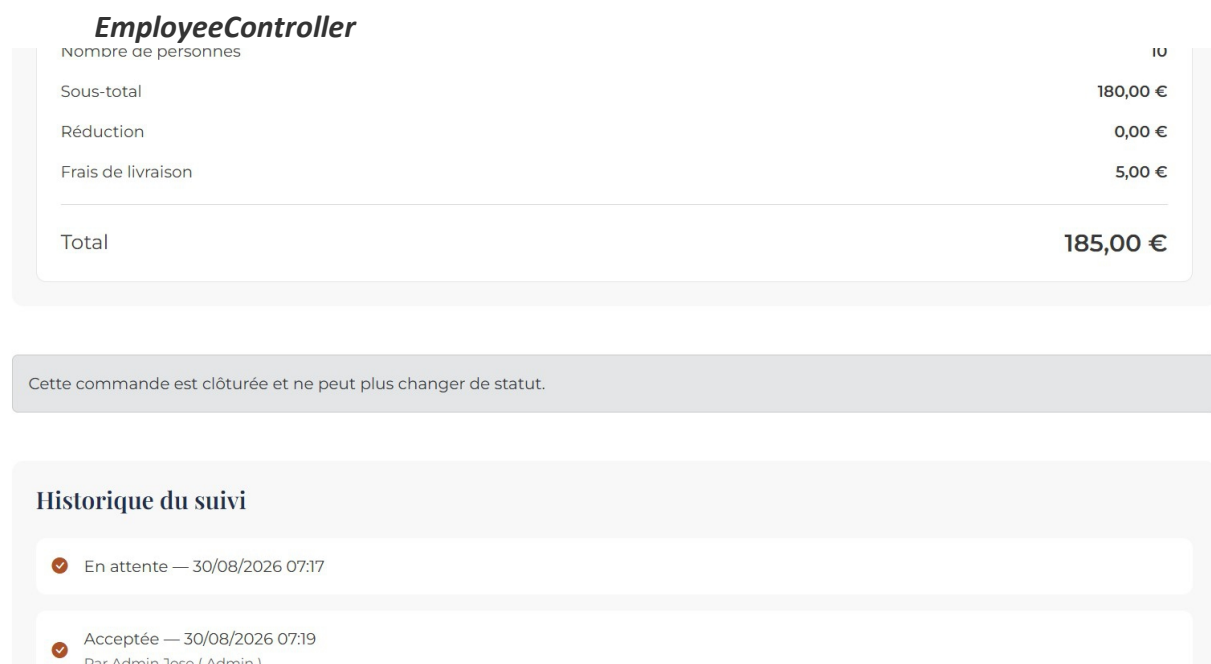


Figure 21: Affichage de l'historique des changements de statut d'une commande

3.8 Gestion des e-mails transactionnels

L'application intègre un système d'envoi d'e-mails transactionnels afin d'informer les utilisateurs lors des principales étapes de leur utilisation du service. Les envois sont réalisés à l'aide de l'API Brevo et leur gestion est centralisée dans la classe **MailService**.

Cette classe regroupe les différents types d'e-mails utilisés par l'application : message de bienvenue lors de la création d'un compte client, confirmation d'une commande, réinitialisation du mot de passe, création d'un compte employé, transmission des demandes du formulaire de contact, demande de retour du matériel prêté et invitation à déposer un avis lorsqu'une commande est terminée.

Chaque type d'e-mail dispose d'une méthode dédiée. Le contenu HTML n'est pas directement écrit dans ces méthodes : il est généré à partir de vues PHP situées dans le dossier **Views/emails**. La méthode **renderView()** reçoit le nom du modèle d'e-mail ainsi que les données nécessaires, puis utilise la temporisation de sortie PHP pour récupérer le contenu HTML généré.

L'envoi proprement dit est centralisé dans la méthode privée **send()**. Celle-ci construit la requête contenant notamment le destinataire, l'expéditeur, l'objet et le contenu HTML, puis utilise le client Brevo pour transmettre le message. Cette centralisation évite de répéter la configuration de l'envoi dans chaque fonctionnalité.

Les informations de configuration nécessaires à Brevo sont récupérées depuis un fichier de configuration dédié. Les informations sensibles, notamment les identifiants permettant d'utiliser le service externe, ne sont ainsi pas directement inscrites dans les contrôleurs ou dans les vues de l'application.

Les e-mails sont déclenchés par les actions métier correspondantes. Par exemple, après la création d'une commande, **OrderController** appelle **sendOrderConfirmationEmail()**. Lorsqu'une commande avec prêt de matériel atteint le statut d'attente du retour, **sendEquipmentReturnEmail()** est utilisé. Enfin, lorsque la commande passe au statut terminé, **sendReviewInvitationEmail()** transmet au client un lien lui permettant d'accéder à sa commande et de déposer un avis.

La méthode d'envoi est entourée d'un bloc **try/catch**. En cas d'échec de la communication avec le service d'e-mail, l'erreur est enregistrée dans les journaux de l'application et la méthode retourne false, ce qui permet d'éviter qu'une erreur du service externe provoque directement l'arrêt de l'application.

```

private function send(
    string $recipientEmail,
    string $recipientName,
    string $subject,
    string $htmlContent
): bool {
    try {
        $email = new SendTransacEmailRequest([ ...
    ]);

        $this->client
            ->transactionalEmails
            ->sendTransacEmail($email);

        return true;
    } catch (\Throwable $exception) {
        error_log(
            'Erreur lors de l'envoi du mail Brevo : '
            . $exception->getMessage()
        );

        return false;
    }
}

```

Figure 22: Centralisation de l'envoi des e-mails transactionnels dans *MailService*



Vite & Gourmand <contact@vite-et-gourmand.dev>

À moi ▼

Bonjour yoann modif,

Votre commande a bien été enregistrée.

Elle est actuellement en attente de validation.

Récapitulatif

- **Commande** : #13
- **Menu** : Buffet Signature Réception
- **Nombre de personnes** : 10
- **Date de livraison** : 12/09/2026
- **Heure** : 07:30
- **Adresse** : 8 rue d'inkermann
- **Ville** : Bordeaux
- **Montant total** : 185,00 €
- **Prêt de matériel** : Non

Figure 23: Exemple d'e-mail transactionnel envoyé par l'application

3.9 Gestion des avis clients

L'application permet aux clients de déposer un avis à la suite d'une commande terminée. Afin de garantir que les avis proviennent de clients ayant réellement utilisé le service, le dépôt d'un avis est directement lié à une commande et à l'utilisateur connecté.

La gestion de cette fonctionnalité repose principalement sur le **ReviewController**, le

ReviewModel ainsi que sur le **EmployeeReviewController** pour la partie modération. Lorsqu'un client souhaite laisser un avis, le contrôleur vérifie tout d'abord, à l'aide de la méthode **canLeaveReview()**, qu'il est autorisé à effectuer cette action.

Cette vérification impose deux conditions. La commande doit appartenir à l'utilisateur connecté et avoir atteint le statut terminé, correspondant à l'identifiant de statut 7. Une seconde requête vérifie qu'aucun avis n'existe déjà pour cette commande. Ce fonctionnement empêche ainsi un utilisateur de déposer un avis sur la commande d'un autre client, sur une commande non terminée ou de publier plusieurs avis pour une même commande.

Lors de la soumission du formulaire, la note est également contrôlée afin d'être comprise entre 1 et 5 et le commentaire ne peut pas être vide. Lorsque ces conditions sont respectées, la méthode **createReview()** enregistre l'avis dans la base de données. Celui-ci est créé avec la valeur **est_valide** définie à 0 et n'est donc pas immédiatement considéré comme validé.

Les avis en attente sont accessibles depuis l'espace employé. La méthode **getPendingReviews()** récupère les avis dont la valeur **est_valide** est égale à 0 ainsi que les informations nécessaires concernant le client et la commande associée. L'accès à cette fonctionnalité est protégé par **Auth::requireRole(['Employé', 'Admin'])**.

L'employé ou l'administrateur peut ensuite accepter ou refuser un avis. La validation est réalisée par la méthode **validateReview()**, qui fait passer **est_valide** à 1. En cas de refus, la méthode **deleteReview()** supprime l'avis de la base de données. Les mêmes contrôles de rôle sont appliqués aux actions de validation et de suppression.

Enfin, la méthode **getValidatedReviews()** permet de récupérer uniquement les avis ayant été validés. Dans l'implémentation actuelle, les quatre derniers avis validés peuvent ainsi être récupérés pour être présentés aux visiteurs de l'application.

```

public function canLeaveReview(int $orderId, int $userId): bool
{
    $sql = "
        SELECT COUNT(*)
        FROM commande
        WHERE id = ?
        AND utilisateur_id = ?
        AND statut_id = 7
    ";

    $stmt = $this->pdo->prepare($sql);
    $stmt->execute([$orderId, $userId]);

    $orderExists = (int) $stmt->fetchColumn();

    if ($orderExists === 0) {
        return false;
    }

    $sql = "
        SELECT COUNT(*)
        FROM avis
        WHERE commande_id = ?
    ";

    $stmt = $this->pdo->prepare($sql);
    $stmt->execute([$orderId]);

    $reviewExists = (int) $stmt->fetchColumn();

    return $reviewExists === 0;
}

```

Figure 24: Contrôle des conditions autorisant le dépôt d'un avis

Gestion des avis clients

Validez ou refusez les avis reçus avant leur affichage sur la page d'accueil.

Avis en attente

yoann modif Lamouille

En attente

Commande #CMD-14



Avis 5 étoiles pour mon document technique

Valider

Refuser

Figure 25: Modération des avis clients depuis l'espace employé

3.10 Gestion des utilisateurs et des employés

L'application distingue trois types d'utilisateurs : les clients, les employés et l'administrateur. Ces différents comptes sont enregistrés dans la même table **utilisateur** et sont associés à un rôle permettant de déterminer les fonctionnalités auxquelles ils peuvent accéder.

La création d'un compte client est réalisée depuis **AuthController**. Avant l'enregistrement, les informations saisies sont contrôlées à l'aide de l'entité **Utilisateur**. Celle-ci permet notamment de vérifier la présence des informations obligatoires, la validité de l'adresse e-mail, le format du numéro de téléphone, la longueur des données ainsi que la robustesse du mot de passe. Le mot de passe doit comporter au minimum dix caractères, avec une majuscule, une minuscule, un chiffre et un caractère spécial. Avant la création du compte, l'application vérifie également que l'adresse e-mail n'est pas déjà utilisée. Le mot de passe n'est jamais enregistré directement dans

la base de données : il est transformé à l'aide de **password_hash()** avec **PASSWORD_DEFAULT**, puis seul le hash obtenu est enregistré.

Après la création réussie d'un compte client, une session utilisateur est créée et son identifiant est régénéré. Un e-mail de bienvenue est également envoyé par l'intermédiaire du service d'e-mails transactionnels présenté précédemment.

La gestion des comptes employés est réservée à l'administrateur. Depuis son espace d'administration, celui-ci peut consulter la liste des employés, créer un nouveau compte, modifier ses informations ou modifier son état actif.

Lors de la création d'un employé, les mêmes règles principales de validation sont appliquées concernant le nom, le prénom, l'adresse e-mail et le mot de passe. Le compte créé reçoit automatiquement le rôle Employé et est actif par défaut. Après sa création, un e-mail informe l'employé de la création de son compte.

L'administrateur peut également modifier le nom, le prénom et l'adresse e-mail d'un employé. Avant la modification, l'application vérifie notamment que la nouvelle adresse e-mail n'est pas déjà associée à un autre utilisateur.

Plutôt que de supprimer un compte employé, l'application permet de l'activer ou de le désactiver.

La méthode **toggleEmployeeStatus()** inverse son état actuel puis **UserModel::updateEmployeeStatus()** enregistre cette modification. Un employé désactivé reste ainsi présent dans la base de données mais ne peut plus se connecter à l'application. Ce fonctionnement permet de conserver les données et les références historiques associées à son compte, notamment les actions enregistrées dans le suivi des commandes.

L'application propose également une procédure de réinitialisation du mot de passe. Lorsqu'une demande est effectuée, un jeton aléatoire est généré avec **random_bytes()**. Seul son hash SHA256 est conservé côté serveur et le lien transmis par e-mail contient le jeton original. Le lien possède une durée de validité d'une heure. Après utilisation et modification du mot de passe, les informations de réinitialisation associées à l'utilisateur sont supprimées.

```
$passwordHash = password_hash(
    $password,
    PASSWORD_DEFAULT
);

$employeeCreated = $userModel->createEmployee(
    $nom,
    $prenom,
    $email,
    $passwordHash
);
```

Figure 26: Validation et création sécurisée d'un compte employé

Gestion des employés

Créez, consultez ou désactivez les comptes employés.

Employés existants

Créer un employé

Nom	Email	Rôle	Statut	Actions
employee yoann	yoann.lamouille@hotmail.fr	Employé	Actif	Modifier Désactiver

Figure 27: Gestion des comptes employés depuis l'espace administrateur

4 Sécurité de l'application

La sécurité constitue un élément important du développement de l'application Vite & Gourmand. Plusieurs mécanismes ont été mis en place afin de protéger les comptes utilisateurs, contrôler l'accès aux différentes fonctionnalités, sécuriser les données manipulées par l'application et éviter l'exposition d'informations sensibles. Cette partie présente les principales mesures de sécurité intégrées au projet, de l'authentification jusqu'à la gestion des fichiers et de la configuration de l'environnement.

4.1 Sécurisation de l'authentification et des mots de passe

L'authentification de l'application repose sur une adresse e-mail et un mot de passe. Plusieurs contrôles sont effectués côté serveur afin de limiter les données incorrectes et de protéger les comptes utilisateurs.

Lors de la création d'un compte client ou employé, le mot de passe doit respecter une politique de complexité définie dans l'entité **Utilisateur**. Il doit contenir au minimum dix caractères, dont au moins une

lettre minuscule, une lettre majuscule, un chiffre et un caractère spécial. La confirmation du mot de passe est également comparée au mot de passe saisi avant la création du compte.

Les mots de passe ne sont jamais enregistrés en clair dans la base de données. Avant leur stockage, ils sont traités avec la fonction PHP **password_hash()** et l'algorithme sélectionné par **PASSWORD_DEFAULT**. La base de données conserve uniquement le hash obtenu.

Lors de la connexion, l'utilisateur est recherché à partir de son adresse e-mail. La fonction **password_verify()** compare ensuite le mot de passe transmis avec le hash enregistré. L'application ne réalise donc pas de comparaison avec un mot de passe stocké en clair.

En cas d'adresse e-mail inexistante ou de mot de passe incorrect, le même message « Email ou mot de passe incorrect. » est retourné. Cette réponse volontairement générique évite d'indiquer à une personne extérieure si une adresse e-mail correspond ou non à un compte existant.

L'état du compte est également contrôlé après la vérification des identifiants. Un compte désactivé ne peut pas ouvrir de session, même lorsque le mot de passe fourni est correct. Ce mécanisme est notamment utilisé pour permettre à l'administrateur de désactiver l'accès d'un employé sans supprimer son compte et les données qui lui sont associées.

Après une authentification réussie, l'identifiant de session PHP est régénéré avec **session_regenerate_id(true)** avant l'enregistrement des informations de l'utilisateur dans la session. Cette mesure participe à la protection de la session après la connexion. La gestion plus complète de la durée de session et des autorisations sera présentée dans la section suivante.

```
$userModel = new UserModel();
$user = $userModel->findByEmail($email);

if (
    !$user ||
    !password_verify(
        $password,
        $user['mot_de_passe_hash']
    )
) {
    $error = "Email ou mot de passe incorrect.";
} else {

    $utilisateur = new Utilisateur(
        $user['nom'],
        $user['prenom'],
        $user['email'],
        (bool) $user['actif'],
        $user['role']
    );

    if (!$utilisateur->estActif()) {
        $error = "Ce compte est désactivé.";
    } else {

        session_regenerate_id(true);
```

Figure 28: Vérification du mot de passe et sécurisation de l'ouverture de session

4.2 Gestion sécurisée des sessions et des autorisations

L'application utilise les sessions PHP pour conserver l'état de connexion de l'utilisateur. Lors d'une authentification réussie, l'identifiant de session est régénéré avec `session_regenerate_id(true)` afin de limiter les risques de fixation de session. Les informations nécessaires à l'identification de l'utilisateur, notamment son identifiant et son rôle, sont ensuite enregistrées dans la session. Une durée maximale d'inactivité de 30 minutes est appliquée aux espaces protégés. La classe **Auth** conserve la date de dernière activité de l'utilisateur et vérifie cette valeur lors des contrôles d'accès. Lorsque cette durée est dépassée, la session est détruite et l'utilisateur est redirigé vers la page de connexion.

Les autorisations sont contrôlées avec la méthode **Auth::requireRole()**. Celle-ci vérifie d'abord qu'un utilisateur est connecté, puis récupère ses informations directement depuis la base de données. Cela permet notamment de vérifier que son compte est toujours actif et que son rôle correspond aux rôles autorisés pour la ressource demandée.

Ce contrôle est utilisé dans les différents contrôleurs de l'application. Par exemple, les fonctionnalités de gestion des commandes sont accessibles aux employés et aux administrateurs avec

Auth::requireRole(['Employé', 'Admin']), tandis que les statistiques et la gestion des employés sont réservées au rôle **Admin**.

Cette organisation permet de centraliser les contrôles d'accès et d'éviter de se limiter à l'affichage ou au masquage des fonctionnalités dans l'interface : l'autorisation est également vérifiée côté serveur avant l'exécution des fonctionnalités protégées.

```
class Auth
{
    private const SESSION_TIMEOUT = 1800; // 30 minutes

    private static function startSession(): void
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
    }

    private static function checkSessionTimeout(): void
    {
        if (!isset($_SESSION['user'])) {
            return;
        }

        if (
            isset($_SESSION['last_activity']) &&
            time() - $_SESSION['last_activity'] > self::SESSION_TIMEOUT
        ) {
            $_SESSION = [];
            session_destroy();

            header('Location: index.php?url=connexion&session=expired');
            exit;
        }

        $_SESSION['last_activity'] = time();
    }
}
```

Figure 29: Gestion de la session et expiration après 30 minutes d'inactivité

```

public static function requireRole(array $roles): void
{
    header('Location: index.php?url=connexion');
    exit;
}

$userModel = new UserModel();

$user = $userModel->findById(
    (int) $_SESSION['user']['id']
);

if (!$user) {
    $_SESSION = [];
    session_destroy();

    header('Location: index.php?url=connexion');
    exit;
}

$utilisateur = new Utilisateur(
    $user['nom'],
    $user['prenom'],
    $user['email'],
    (bool) $user['actif'],
    $user['role']
);

if (!$utilisateur->estActif()) {
    $_SESSION = [];
    session_destroy();

    header('Location: index.php?url=connexion');
    exit;
}

if (!$utilisateur->aUnDesRoles($roles)) {
    header('Location: index.php');
    exit;
}

```

Figure 30: Contrôle de l'authentification, de l'état du compte et du rôle de l'utilisateur

4.3 Sécurisation des données saisies et des requêtes SQL

Les données provenant des formulaires et des paramètres transmis à l'application font l'objet de différents contrôles avant leur utilisation. Selon leur nature, les valeurs sont nettoyées avec **trim()**, converties vers le type attendu ou contrôlées à l'aide de règles de validation.

Les adresses e-mail sont notamment vérifiées avec **filter_var()** et les contraintes propres à certaines données, comme le numéro de téléphone ou le mot de passe, sont regroupées dans l'entité **Utilisateur**. Des contrôles supplémentaires sont réalisés dans les contrôleurs afin de vérifier la présence des champs obligatoires et la cohérence des données avant leur transmission aux modèles.

Pour les échanges avec MySQL, l'application utilise PDO et privilégie les requêtes préparées avec **prepare()** et **execute()**. Les données provenant de l'utilisateur ne sont ainsi pas directement concaténées dans les requêtes SQL. Cette pratique permet notamment de limiter les risques d'injection SQL.

Lors de l’affichage de données dynamiques dans les vues, **htmlspecialchars()** est utilisé pour encoder les caractères HTML lorsque cela est nécessaire. Cette précaution permet de limiter l’interprétation de contenu HTML ou JavaScript injecté dans les données affichées et contribue ainsi à la protection contre les attaques de type XSS.

Ces différents contrôles sont réalisés côté serveur et ne reposent donc pas uniquement sur les contraintes HTML présentes dans les formulaires.

```
$nom = trim($_POST['nom'] ?? '');
$prenom = trim($_POST['prenom'] ?? '');
$email = trim($_POST['email'] ?? '');
$telephone = trim($_POST['telephone'] ?? '');
$adresse = trim($_POST['adresse'] ?? '');
$password = $_POST['password'] ?? '';
$confirmPassword = $_POST['confirm_password'] ?? '';

$utilisateur = new Utilisateur(
    $nom,
    $prenom,
    $email,
    true,
    'Client'
);
if (
    !$utilisateur->informationsValides() ||
    $telephone === '' ||
    $adresse === '' ||
    $password === '' ||
    $confirmPassword === ''
) {
    $error = "Tous les champs sont obligatoires.";
} elseif (!$utilisateur->longueursValides()) {
    $error = "Un ou plusieurs champs sont trop longs.";
} elseif (!$utilisateur->emailValide()) {
    $error = "Adresse email invalide.";
} elseif (!$utilisateur::telephoneValide($telephone)) {
    $error = "Numéro de téléphone invalide.";
} elseif ($password !== $confirmPassword) {
    $error = "Les mots de passe ne correspondent pas.";
} else {
    if (!$utilisateur::motDePasseValide($password)) {
        $error = "Le mot de passe doit contenir au moins 10 caractères, une majuscule, une minuscule, un chiffre et un caractère spécial.";
    } else {
        $userModel = new UserModel();

        if ($userModel->emailExists($email)) {
            $error = "Cette adresse email existe déjà.";
        } else {
```

Figure 31: Validation côté serveur des données saisies lors de l’inscription

```

1 reference | 0 overrides
public function createUser(
    string $nom,
    string $prenom,
    string $email,
    string $telephone,
    string $adresse,
    string $passwordHash
): bool {
    $pdo = Database::getConnection();

    $sql = "
        INSERT INTO utilisateur (
            nom,
            prenom,
            email,
            telephone,
            adresse,
            mot_de_passe_hash,
            date_inscription,
            role_id
        )
        VALUES (?, ?, ?, ?, ?, ?, ?, NOW(), 1)
    ";

    $stmt = $pdo->prepare($sql);

    return $stmt->execute([
        $nom,
        $prenom,
        $email,
        $telephone,
        $adresse,
        $passwordHash
    ]);
}

```

Figure 32: Utilisation d'une requête préparée PDO pour l'enregistrement des données

4.4 Protection des données sensibles et configuration de l'environnement

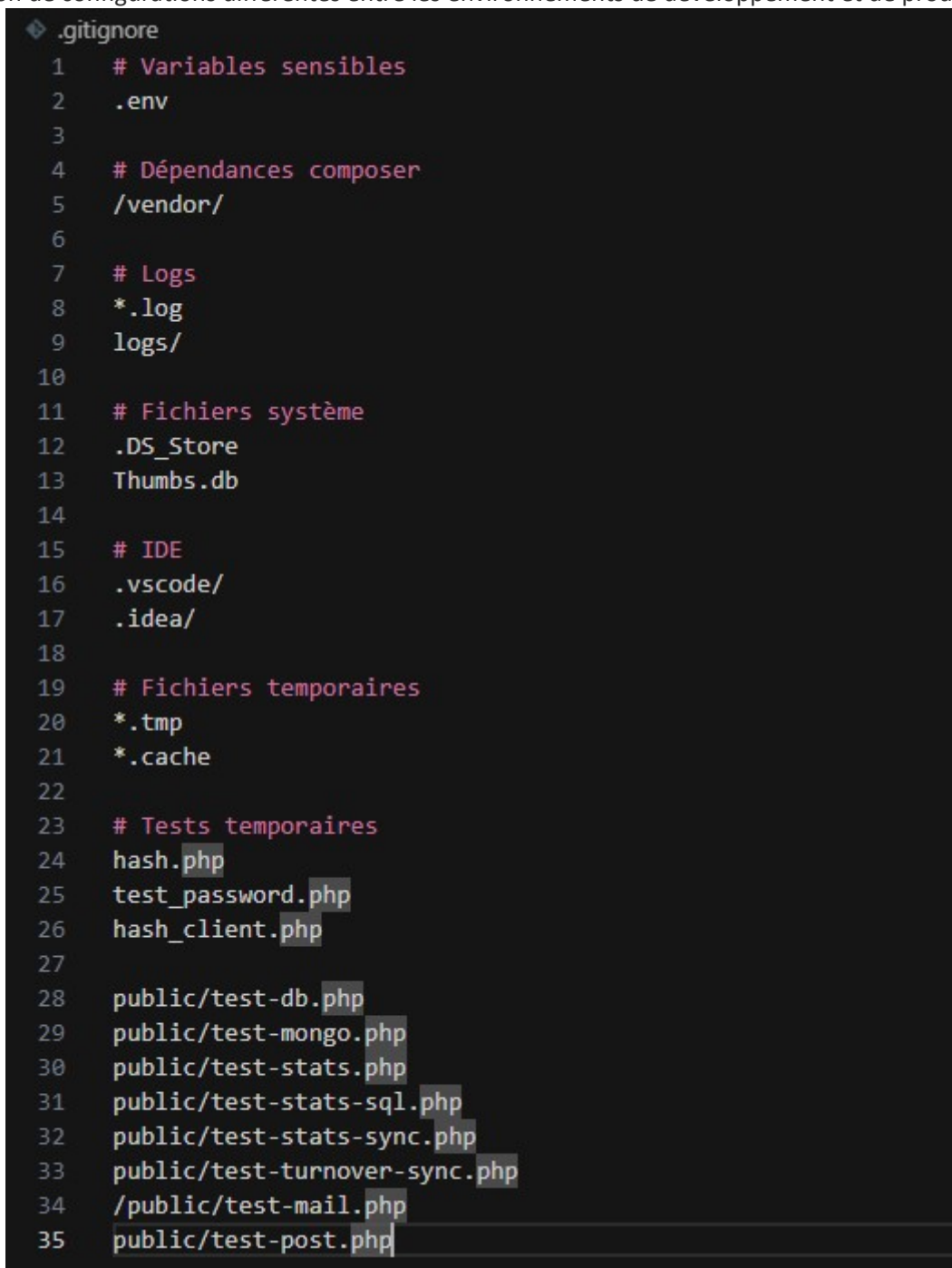
Les informations sensibles nécessaires au fonctionnement de l'application ne sont pas enregistrées directement dans le code source. Elles sont définies à l'aide de variables d'environnement, ce qui permet de séparer la configuration de l'application de son code. Cette organisation concerne notamment les identifiants de connexion à la base de données MySQL, la configuration de MongoDB ainsi que les clés nécessaires aux services externes tels que Brevo.

En environnement local, ces paramètres peuvent être définis dans un fichier .env. Ce fichier contient des informations sensibles et n'a pas vocation à être versionné avec le reste du projet. Il est donc exclu du dépôt Git à l'aide du fichier .gitignore.

Dans l'environnement Docker, les variables nécessaires sont transmises aux conteneurs à partir de la configuration définie dans `compose.yaml`. En production, les valeurs propres à l'environnement de déploiement sont configurées séparément, sans avoir à modifier le code source.

Les classes chargées des connexions récupèrent ensuite ces informations depuis l'environnement. La classe `Database` utilise par exemple les variables de connexion MySQL, tandis que `DatabaseMongo` récupère la configuration nécessaire à la connexion MongoDB.

Cette organisation limite ainsi l'exposition d'informations sensibles dans le dépôt de code et facilite l'utilisation de configurations différentes entre les environnements de développement et de production.



```
.gitignore
1  # Variables sensibles
2  .env
3
4  # Dépendances composer
5  /vendor/
6
7  # Logs
8  *.log
9  logs/
10
11 # Fichiers système
12 .DS_Store
13 Thumbs.db
14
15 # IDE
16 .vscode/
17 .idea/
18
19 # Fichiers temporaires
20 *.tmp
21 *.cache
22
23 # Tests temporaires
24 hash.php
25 test_password.php
26 hash_client.php
27
28 public/test-db.php
29 public/test-mongo.php
30 public/test-stats.php
31 public/test-stats-sql.php
32 public/test-stats-sync.php
33 public/test-turnover-sync.php
34 /public/test-mail.php
35 public/test-post.php
```

Figure 33: Exclusion du fichier contenant les variables d'environnement du dépôt Git

```

) use ($envFile): ?string {
    $systemValue = getenv($key);

    if ($systemValue != false) {
        return $systemValue;
    }

    return $envFile[$key] ?? $default;
};

$jawsDbUrl = getenv('JAWSDB_URL');

if ($jawsDbUrl) {
    $parts = parse_url($jawsDbUrl);

    $host = $parts['host'] ?? null;
    $port = (string) ($parts['port'] ?? 3306);
    $dbname = isset($parts['path'])
        ? ltrim($parts['path'], '/')
        : null;
    $user = $parts['user'] ?? null;
    $password = $parts['pass'] ?? null;
} else {
    $host = $getEnvValue('DB_HOST', 'localhost');
    $port = $getEnvValue('DB_PORT', '3306');
    $dbname = $getEnvValue('DB_NAME');
    $user = $getEnvValue('DB_USER');
    $password = $getEnvValue('DB_PASSWORD');
}

```

Figure 34: Récupération des paramètres de connexion à partir des variables d'environnement

4.5 Sécurisation des fichiers téléversés

L'application permet l'envoi d'images lors de la création ou de la modification d'un menu. Afin de limiter les risques liés au téléversement de fichiers, plusieurs contrôles sont réalisés avant leur enregistrement sur le serveur.

Les fichiers sont récupérés depuis `$_FILES` puis leur état d'envoi est vérifié à l'aide de la constante **UPLOAD_ERR_OK**. Le type **MIME** réel du fichier est ensuite contrôlé avec **mime_content_type()**.

Seuls les formats JPEG, PNG et WebP sont autorisés. Les extensions utilisées pour les fichiers enregistrés sont déterminées à partir du type MIME validé et ne reposent donc pas uniquement sur le nom fourni par l'utilisateur.

Le nom final du fichier est généré par l'application selon son ordre d'affichage, par exemple **image-1.jpg** ou **image-2.webp**. Le fichier est ensuite déplacé dans le répertoire correspondant au menu à l'aide de **move_uploaded_file()**.

Le nombre d'images est également limité lors de la création et de la modification d'un menu. Ces différents contrôles permettent de réduire le risque d'enregistrement de fichiers non autorisés dans les répertoires publics de l'application.

```

        $allowedTypes = [
            'image/jpeg' => 'jpg',
            'image/png' => 'png',
            'image/webp' => 'webp'
        ];

        $themeSlug = $this->getThemeFolder($theme);
        $menuSlug = $menuId . '-' . $this->slugify($titre);

        $relativeDir = "assets/images/menus/$themeSlug/$menuSlug";
        $absoluteDir = __DIR__ . "/../../public/$relativeDir";

        if (!is_dir($absoluteDir)) {
            mkdir($absoluteDir, 0777, true);
        }

        $pdo = Database::getConnection();

        foreach ($images['name'] as $index => $name) {
            if ($images['error'][$index] !== UPLOAD_ERR_OK) {
                return false;
            }

            $tmpName = $images['tmp_name'][$index];
            $mimeType = mime_content_type($tmpName);

            if (!isset($allowedTypes[$mimeType])) {
                return false;
            }

            $order = $index + 1;
            $extension = $allowedTypes[$mimeType];

            $fileName = "image-$order.$extension";
            $relativePath = "$relativeDir/$fileName";
            $absolutePath = "$absoluteDir/$fileName";

            if (!move_uploaded_file($tmpName, $absolutePath)) {
                return false;
            }
        }
    }
}

```

Figure 35: Contrôle du type MIME et enregistrement sécurisé des images téléversées

4.6 Réinitialisation sécurisée du mot de passe

L'application propose une procédure de réinitialisation du mot de passe permettant à un utilisateur de retrouver l'accès à son compte sans exposer directement ses identifiants.

Lorsqu'une demande est effectuée, l'adresse e-mail saisie est contrôlée puis l'utilisateur correspondant est recherché dans la base de données. Afin de ne pas révéler l'existence ou non d'un compte, le message affiché après la demande reste volontairement générique. Si un compte correspondant est trouvé, un jeton aléatoire est généré à l'aide de

random_bytes(32). Ce jeton n'est pas stocké directement en base de données. Un hash SHA-256 est créé avec hash('sha256', \$token) et seul ce hash est enregistré.

Une date d'expiration est également associée à la demande. Dans l'implémentation actuelle, le lien de réinitialisation est valable pendant une heure. Le jeton original est uniquement intégré dans le lien transmis à l'utilisateur par e-mail.

Lorsqu'un utilisateur ouvre ce lien, le jeton reçu est de nouveau haché puis recherché parmi les demandes encore valides. Si aucun enregistrement correspondant n'est trouvé ou si la demande a expiré, la modification du mot de passe est refusée.

Le nouveau mot de passe doit respecter les mêmes règles de complexité que lors de la création d'un compte. Il est ensuite transformé avec password_hash() avant d'être enregistré dans la base de données.

Après une modification réussie, la demande de réinitialisation associée à l'utilisateur est supprimée afin d'empêcher la réutilisation du même lien.

Ce mécanisme permet ainsi de limiter la durée de validité des liens, d'éviter de conserver les jetons de réinitialisation en clair et de garantir que les nouveaux mots de passe sont enregistrés sous forme de hash.

```
if ($user) {  
    $token = bin2hex(random_bytes(32));  
  
    $tokenHash = hash(  
        'sha256',  
        $token  
    );  
  
    $expirationDate = date(  
        'Y-m-d H:i:s',  
        time() + 3600  
    );  
}
```

Figure 36: Génération, hachage et limitation de durée du jeton de réinitialisation

```

$tokenHash = hash('sha256', $token);

$passwordResetModel = new PasswordResetModel();

$reset = $passwordResetModel->findValidByTokenHash(
    $tokenHash
);

if (!$reset) {
    $error = "Ce lien de réinitialisation est invalide ou a expiré.";
    require_once __DIR__ . '/../Views/pages/reset-password.php';
    return;
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    $password = $_POST['password'] ?? '';
    $confirmPassword = $_POST['confirm_password'] ?? '';

    if (
        $password === ''
        || $confirmPassword === ''
    ) {
        $error = "Tous les champs sont obligatoires.";
    } elseif ($password !== $confirmPassword) {
        $error = "Les mots de passe ne correspondent pas.";
    } else {
        if (!Utilisateur::motDePasseValide($password)) {
            $error = "Le mot de passe doit contenir au moins 10 caractères, une majuscule, une minuscule, un chiffre et un caractère spécial.";
        } else {
            $userModel = new UserModel();

            $passwordHash = password_hash(
                $password,
                PASSWORD_DEFAULT
            );

```

Figure 37: Vérification du jeton et enregistrement sécurisé du nouveau mot de passe

5 Tests et validation de l'application

Afin de vérifier le bon fonctionnement de l'application et de limiter les régressions, différents tests ont été réalisés au cours du développement. Ils concernent aussi bien les fonctionnalités accessibles aux utilisateurs que les règles métier et la sécurité de l'application.

Des tests unitaires automatisés ont été mis en place avec **PHPUnit** afin de vérifier les principales règles métier indépendamment de la base de données. Ils portent sur les entités **Commande**, **Menu** et **Utilisateur**. Ils permettent notamment de contrôler le calcul du prix d'une commande, l'application de la réduction, les frais de livraison, le respect du nombre minimum de personnes, la disponibilité du stock ainsi que certaines règles de validation des comptes utilisateurs.

En complément, des tests manuels ont été réalisés tout au long du développement afin de vérifier les parcours nécessitant des interactions avec l'application, notamment l'inscription et la connexion, la gestion des commandes, les différents rôles, les e-mails transactionnels ainsi que les échanges avec les bases de données MySQL et MongoDB.

5.1 Tests unitaires avec PHPUnit

Des tests unitaires automatisés ont été réalisés avec **PHPUnit** afin de vérifier le comportement des principales règles métier de l'application de manière isolée. PHPUnit est installé comme dépendance de développement via Composer.

Les tests sont regroupés dans le dossier **tests** et concernent trois entités : **Commande**, **Menu** et **Utilisateur**. Ils ne nécessitent aucune connexion aux bases de données MySQL ou MongoDB et n'ajoutent donc aucune donnée lors de leur exécution.

Les tests de l'entité **Commande** vérifient notamment le calcul du sous-total, l'application ou non de la réduction de 10 %, le calcul des frais de livraison et le montant total. Les tests de Menu contrôlent notamment le nombre minimum de personnes, le stock disponible et la validité des données. Enfin, les

tests de Utilisateur vérifient entre autres l'état du compte, les rôles, le format de l'adresse e-mail, du numéro de téléphone et les règles de validation du mot de passe.

```
0 references | 0 implementations
class CommandeTest extends TestCase
{
    0 references | 0 overrides
    public function testCalculSousTotal(): void
    {
        $commande = new Commande(
            10,
            18.00,
            5,
            0
        );

        $this->assertSame(
            180.0,
            $commande->calculerSousTotal()
        );
    }

    0 references | 0 overrides
    public function testReductionAppliquee(): void
    {
        $commande = new Commande(
            10,
            18.00,
            5,
            0
        );

        $this->assertSame(
            18.0,
            $commande->calculerReduction()
        );
    }
}
```

Figure 38: Exemple de tests unitaires de l'entité Commande avec PHPUnit

5.2 Exécution et résultats des tests

L'ensemble des tests unitaires peut être exécuté simultanément depuis le terminal à l'aide de la commande **vendor/bin/phpunit tests**. PHPUnit détecte alors les différents fichiers de tests présents dans le dossier tests et exécute automatiquement les méthodes de test.

Lors de l'exécution finale, 29 tests correspondant à 29 assertions ont été exécutés avec succès. Le résultat obtenu est de 100 % de réussite, sans erreur ni échec. Cette exécution permet notamment de vérifier rapidement que les règles métier testées continuent de fonctionner après une modification du code.

Ces tests constituent ainsi un moyen de limiter les régressions sur les comportements couverts par la suite de tests.

```
PS C:\wamp64\www\vite-et-gourmand-ecf> vendor\bin\phpunit tests
PHPUnit 13.3.2 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.4.15

.....                                              29 / 29 (100%)

Time: 00:00.027, Memory: 20.00 MB

OK (29 tests, 29 assertions)
PS C:\wamp64\www\vite-et-gourmand-ecf> git status
```

Figure 39: Exécution de l'ensemble des tests unitaires avec PHPUnit

5.3 Tests fonctionnels manuels

En complément des tests unitaires automatisés, des tests fonctionnels manuels ont été réalisés régulièrement tout au long du développement. Ils ont permis de vérifier les différents parcours de l'application avec les rôles Client, Employé et Administrateur.

Les tests ont notamment porté sur l'inscription et la connexion, la gestion du profil, la réinitialisation du mot de passe, la création, la modification et l'annulation des commandes, leur changement de statut, la gestion des menus et des plats, la modération des avis, les fonctionnalités d'administration ainsi que l'envoi des e-mails transactionnels.

Ces vérifications ont également permis d'identifier et de corriger différentes anomalies au cours du développement : erreurs d'affichage et de responsive, problèmes dans certaines vues, erreurs dans l'application de règles métier ou encore problèmes de navigation selon l'environnement d'exécution. Certains liens fonctionnaient par exemple avec Docker mais pas avec WampServer. La gestion des chemins et de l'URL de base de l'application a donc été adaptée afin de permettre son fonctionnement dans les deux environnements.

Les tests des e-mails transactionnels ont également mis en évidence un problème de délivrabilité, certains messages étant initialement classés comme indésirables. Une adresse d'expédition associée au domaine de l'application a donc été mise en place. Le domaine a été authentifié avec une signature DKIM et une politique DMARC, puis de nouveaux tests ont permis de vérifier la réception des messages.

Les tests manuels ont ainsi été utilisés comme un processus itératif : exécution d'un scénario, identification d'une anomalie, correction puis nouveau test. Ils complètent les tests unitaires en vérifiant le fonctionnement réel des parcours et les interactions entre les différentes parties de l'application.

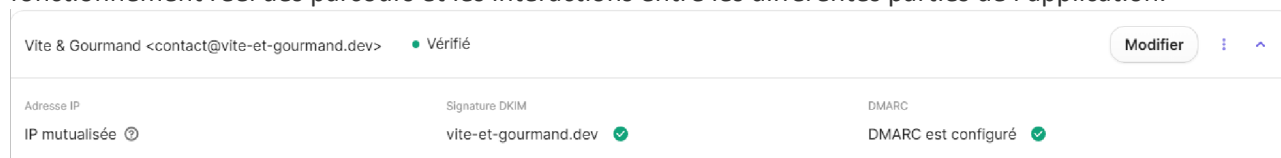


Figure 40: Authentification du domaine utilisé pour l'envoi des e-mails transactionnels

Thème

Tous les thèmes



Réinitialiser les filtres



Buffet Signature Réception

Buffet traiteur complet pour réceptions professionnelles ou privées, équilibré et gourmand.

Min. 10 personnes

Standard

18 € par personne

Voir les détails du menu

Figure 41: Vérification de l'affichage responsive de l'application sur mobile

6 Déploiement et mise en production

Le déploiement de l'application a été préparé afin de permettre son exécution dans différents environnements tout en conservant une configuration cohérente. Le projet peut être exécuté localement avec **WampServer** ou à l'aide de conteneurs **Docker**, puis déployé sur un environnement distant.

La configuration nécessaire au fonctionnement de l'application est séparée du code source grâce à des variables d'environnement. Cette organisation permet notamment d'adapter les connexions aux bases de données, les URL de l'application et les services externes selon l'environnement utilisé, sans modifier directement le code applicatif.

Cette partie présente la conteneurisation du projet avec **Docker**, le processus de déploiement ainsi que les principaux éléments de configuration nécessaires au fonctionnement de l'application en production.

6.1 Environnement local avec WampServer

Le développement de l'application a initialement été réalisé dans un environnement local avec **WampServer** sous Windows. Cet environnement permet de disposer notamment d'un serveur **Apache**, de **PHP** et d'un serveur **MySQL** nécessaires au fonctionnement de l'application.

Le projet est placé dans le répertoire **www** de **WampServer** et est accessible depuis le navigateur en local. La base de données relationnelle **MySQL** est également exécutée localement et administrée pendant le développement depuis les outils fournis avec cet environnement. **WampServer** a été utilisé pour la majeure partie du développement et pour les premiers tests fonctionnels de l'application. Il a notamment permis de développer progressivement les différentes fonctionnalités avant la mise en place de la conteneurisation avec **Docker**.

L'utilisation ultérieure de plusieurs environnements a nécessité de prendre en compte certaines différences de configuration, notamment concernant les chemins et l'URL de base de l'application. La configuration a ainsi été adaptée afin que les liens et les ressources de l'application puissent fonctionner aussi bien dans l'environnement WampServer que dans l'environnement Docker.

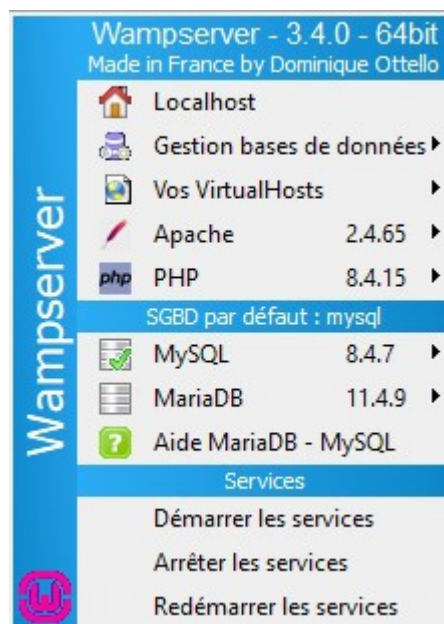


Figure 42: Environnement de développement local avec WampServer

6.2 Conteneurisation avec Docker

Après une première phase de développement avec WampServer, l'application a été conteneurisée avec Docker afin de disposer d'un environnement d'exécution reproductible et moins dépendant de la configuration locale de la machine.

La configuration repose sur un fichier **compose.yaml** qui définit les différents services nécessaires au fonctionnement de l'application. L'environnement Docker comprend notamment un service pour l'application **PHP/Apache**, un service **MySQL** pour la base de données relationnelle et un service MongoDB utilisé pour les statistiques.

Docker Compose permet à ces différents services de fonctionner ensemble au sein du même environnement. Des volumes persistants sont également utilisés pour **MySQL** et **MongoDB** afin de conserver les données lorsque les conteneurs sont arrêtés ou recréés.

Les paramètres dépendant de l'environnement sont définis à l'aide de variables d'environnement. Le fichier **.env** contient notamment les informations nécessaires aux connexions aux bases de données et à la configuration de l'application. Ce fichier est volontairement exclu du dépôt **Git** afin de ne pas versionner les informations sensibles.

La conteneurisation facilite également l'installation du projet sur une autre machine. Après avoir installé **Git** et **Docker**, il faut cloner le dépôt **Git** du projet puis créer et renseigner le fichier **.env** avec les variables nécessaires. Les commandes **Docker** doivent ensuite être exécutées depuis la racine du projet.

L'environnement peut être construit et démarré avec la commande : **docker**

compose up -d --build

Docker Compose construit alors les éléments nécessaires et démarre les différents services définis dans **compose.yaml**. Cette organisation permet de retrouver un environnement cohérent comprenant l'application et ses deux systèmes de gestion de données, sans avoir à reproduire manuellement toute la configuration utilisée pendant le développement.

```

services:
  php:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: vite_et_gourmand_php
    ports:
      - "8080:80"
    volumes:
      - ./var/www/html
    environment:
      APP_BASE_URL: "/public"
      APP_URL: "http://localhost:8080/public"
      DB_HOST: mysql
      DB_PORT: "3306"
      DB_NAME: "${DB_NAME}"
      DB_USER: "${DB_USER}"
      DB_PASSWORD: "${DB_PASSWORD}"

      MONGO_HOST: "${MONGO_HOST}"
      MONGO_PORT: "${MONGO_PORT}"
      MONGO_DATABASE: "${MONGO_DATABASE}"

    depends_on:
      - mysql
      - mongodb

    command: >
      sh -c "composer install --no-interaction --prefer-dist &&
      apache2-foreground"
  mysql:
    image: mysql:8.4
    container_name: vite_et_gourmand_mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: "${MYSQL_ROOT_PASSWORD}"
      MYSQL_DATABASE: "${DB_NAME}"
      MYSQL_USER: "${DB_USER}"
      MYSQL_PASSWORD: "${DB_PASSWORD}"
    ports:
      - "3305:3306"
    volumes:
      - mysql_data:/var/lib/mysql
      - ./database/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro
      - ./database/data.sql:/docker-entrypoint-initdb.d/02-data.sql:ro

```

Figure 43: Configuration des services PHP et MySQL avec Docker Compose

```
mongodb:
  image: mongo:8
  container_name: vite_et_gourmand_mongodb
  restart: unless-stopped
  ports:
    - "27017:27017"
  volumes:
    - mongodb_data:/data/db

volumes:
  mysql_data:
  mongodb_data:
```

Figure 44: Configuration du service MongoDB et des volumes persistants avec Docker Compose

```
[+] up 5/5
✓ Image vite-et-gourmand-ecf-php      Built      11.4s
✓ Network vite-et-gourmand-ecf default Created     0.0s
✓ Container vite_et_gourmand_mongodb  Started    1.0s
✓ Container vite_et_gourmand_mysql    Started    1.0s
✓ Container vite_et_gourmand_php      Started    0.8s

PS C:\wamp64\www\vite-et-gourmand-ecf> docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
vite_et_gourmand_mongodb	mongo:8	"docker-entrypoint.s..."	mongodb	9 seconds ago	Up 8 seconds	0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
vite_et_gourmand_mysql	mysql:8.4	"docker-entrypoint.s..."	mysql	9 seconds ago	Up 8 seconds	0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
vite_et_gourmand_php	vite-et-gourmand-ecf-php	"docker-php-entrypoi..."	php	8 seconds ago	Up 7 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

Figure 45: Exécution des différents conteneurs de l'application

6.3 Déploiement sur Heroku

La version en ligne de l'application est déployée sur **Heroku** ([Vite et Gourmand](#)). Le déploiement permet de rendre l'application accessible depuis Internet tout en utilisant une configuration distincte de l'environnement local.

Le code source est versionné avec **Git** et hébergé sur **GitHub**. Une fois les modifications validées et intégrées dans la branche principale, la version correspondante peut être déployée sur **Heroku**. **Heroku** utilise les fichiers de configuration présents dans le projet pour préparer l'environnement d'exécution de l'application. Les dépendances **PHP** sont installées avec Composer à partir de **composer.json** et **composer.lock**, puis l'application est démarrée à l'aide de la configuration prévue pour la production. Les paramètres propres à l'environnement de production, notamment les connexions aux bases de données et les variables utilisées par les services externes, sont définis directement dans la configuration **Heroku**. Ils ne sont donc pas enregistrés dans le dépôt **Git**.

L'application utilise une base de données **MySQL** distante pour les données relationnelles ainsi qu'un service **MongoDB** distant pour les données statistiques. Les classes de connexion détectent les variables disponibles dans l'environnement de production et utilisent les paramètres correspondants.

Après chaque déploiement, des vérifications fonctionnelles sont réalisées sur la version en ligne afin de contrôler notamment l'accès aux différentes pages, l'authentification, les commandes, les statistiques et l'envoi des e-mails transactionnels.

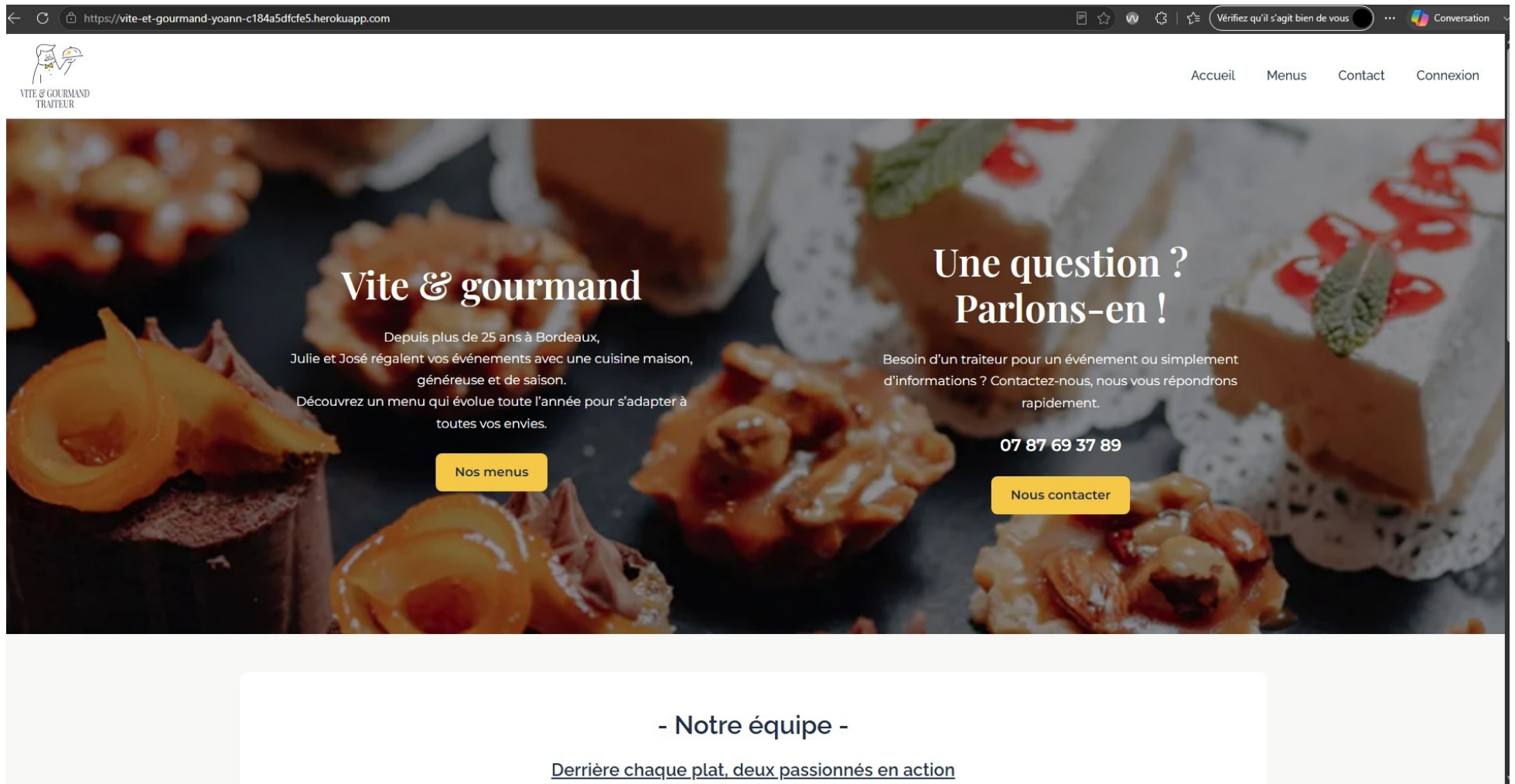


Figure 46: Application déployée et accessible sur Heroku

Key	Value	
APP_BASE_URL		 
APP_URL	https://vite-et-gourmand-yoann-c184a5dfc	 
BREVO_API_KEY		 
CONTACT_EMAIL		 
JAWSDB_URL	mysql://c	 
MAIL_FROM	contact@vite-et-gourmand.dev	 
MAIL_FROM_NAME	Vite & Gourmand	 
MONGO_DATABASE	vite_et_gourmand_stats	 
MONGO_URI	mongodb+srv://	 
KEY	VALUE	

Figure 47: Configuration des variables d'environnement sur Heroku

6.4 Configuration de l'environnement de production

L'environnement de production utilise une configuration distincte de celle employée en local. Les paramètres nécessaires au fonctionnement de l'application sont définis à l'aide de variables d'environnement, ce qui permet d'utiliser le même code source tout en adaptant les connexions aux services distants et les URL selon l'environnement d'exécution.

Pour la base de données relationnelle, l'application utilise en production le service **JawsDB MySQL**, associé à l'application **Heroku**. Ce service fournit une instance **MySQL** distante ainsi que les informations nécessaires à sa connexion. Heroku met notamment à disposition la variable **JAWSDB_URL**, utilisée par l'application pour établir la connexion à la base de production. La structure de la base de données ainsi que les données nécessaires au fonctionnement initial de l'application ont été importées dans cette instance **MySQL**. Les données créées ensuite depuis l'application sont enregistrées dans cette base distante et sont indépendantes des nouveaux déploiements de l'application.

La classe de connexion Database adapte son comportement selon l'environnement. Lorsque **JAWSDB_URL** est disponible, elle récupère les paramètres de connexion à la base **MySQL** de production. En environnement local, elle utilise à la place les variables **DB_HOST**, **DB_PORT**, **DB_NAME**, **DB_USER** et **DB_PASSWORD**.

Le même principe est utilisé pour **MongoDB**, qui stocke les données statistiques de l'application. En production, la connexion à la base **MongoDB** distante est définie par la variable **MONGO_URI** ainsi que par **MONGO_DATABASE**. En environnement **Docker** local, les paramètres correspondent au service **MongoDB** défini dans **Docker Compose**.

Les autres paramètres propres à la production, notamment l'URL publique de l'application et les informations nécessaires à l'envoi des e-mails avec **Brevo**, sont également fournis par les variables

d'environnement configurées sur **Heroku**. Les informations sensibles telles que les mots de passe des bases de données, les chaînes de connexion et la clé **d'API Brevo** ne sont donc pas enregistrées directement dans le code source ni dans le dépôt **Git**.

Cette séparation entre le code et la configuration permet ainsi de conserver une même base de code pour les environnements **WampServer**, **Docker** et **Heroku**, tout en utilisant pour chacun les paramètres et services qui lui sont propres.

The screenshot shows the Heroku dashboard for the application 'vite-et-gourmand-yoann'. The top navigation bar includes 'Overview', 'Resources', 'Deploy', 'Metrics', 'Activity', 'Access', and 'Settings'. The 'Basic Dynos' section shows a 'web' dyno running on 'heroku-php-apache2 public/' with a cost of ~\$0.010/hour. Below this, the 'Add-on Services' section shows a table of services. One service, 'JawsDB MySQL', is listed with attachments, plan, price, and actions. The estimated monthly cost is shown as ~\$7.00/mo.

Service	Attachments	Plan	Price	Actions
JawsDB MySQL jawsdb-cylindrical-20836	Attached as JawsDB	Kitefin Shared	Free	[icon]

Estimated Monthly Cost: ~\$7.00/mo.

Figure 48: Service JawsDB MySQL associé à l'application Heroku

7 Conclusion

La réalisation de l'application Vite & Gourmand a permis de mettre en œuvre l'ensemble des principales étapes nécessaires au développement d'une application web dynamique, depuis la conception de l'architecture et des bases de données jusqu'à son déploiement dans un environnement de production. L'application repose sur une architecture **MVC** en **PHP** et utilise deux systèmes de gestion de données complémentaires : **MySQL** pour les données relationnelles et **MongoDB** pour les statistiques. Les différentes fonctionnalités développées permettent de répondre aux besoins des clients, des employés et de l'administrateur, notamment pour la gestion des menus, des commandes, des utilisateurs, des avis et des e-mails transactionnels.

Une attention particulière a également été portée à la sécurité de l'application, avec la protection de l'authentification et des sessions, la validation des données, l'utilisation de requêtes préparées, la protection des informations sensibles et la sécurisation de la procédure de réinitialisation du mot de passe. La mise en place de tests unitaires avec **PHPUnit** ainsi que les nombreux tests fonctionnels réalisés au cours du développement ont permis de vérifier les principales règles métier et d'identifier puis de corriger différentes anomalies. Enfin, l'utilisation de **Docker** facilite la reproduction de l'environnement d'exécution, tandis que le déploiement sur **Heroku** permet de disposer d'une version de l'application accessible en ligne. Ce projet m'a ainsi permis de mettre en pratique les compétences acquises au cours de ma formation et de conduire le développement d'une application web complète, depuis sa conception jusqu'à sa mise en production.